

LocalHouseLLM Research

INDEPENDENT AI SYSTEMS & LOCAL INFERENCE RESEARCH DIVISION

Master Report: The Complete Local LLM Inference Optimization Compendium

A consolidated six-part technical survey covering quantization, attention kernels, GPU systems history, KV-cache engineering, and emerging/niche acceleration techniques for local large-language-model inference.

Prepared by: LocalHouseLLM Research
Classification: Internal / Partner Technical Compendium
Edition: Master Consolidated Volume — 6 Reports

Table of Contents

01	The Highest-Impact Optimization Layers for Local AI Inference	Core Optimization Survey
02	The Hidden Layer	Niche & Under-the-Radar Techniques
03	Historical Evolution of GPU Inference Optimization	GPU Kernel & Systems History
04	KV Cache Optimization for Large Language Model Inference	KV Cache Deep-Dive Handbook
05	Niche Techniques for Locally Accelerating Large Models	Emerging & Experimental Methods
06	AI Optimization for Local Inference: Highest-Impact Layers	Five-Layer Execution Stack Analysis

Each report retains its original internal structure, ratings, taxonomies, and bibliography. Section dividers mark the beginning of each report within this master volume.

LocalHouseLLM Research

Independent AI Systems & Local Inference Research Division

The Highest-Impact Optimization Layers for Local AI Inference

Report 01 · Core Optimization Survey | Prepared by LocalHouseLLM Research | Confidential Technical Compendium

The Highest-Impact Optimization Layers for Local AI Inference

LocalHouseLLM Research

Executive Summary

Local AI inference now employs a vast array of hardware- and software-level optimizations to achieve low-latency, low-memory inference on edge devices. Key strategies include low-precision quantization, specialized attention kernels, graph compilers, batching and caching schemes, and system-level scheduling. For example, state-of-the-art quantization algorithms (GPTQ, AWQ, SmoothQuant, AQLM, HQQ, BitNet, etc.) compress LLM weights to 4, 2, or even ~ 1.58 bits per parameter, often with minimal accuracy loss. Hardware accelerators like NVIDIA's TensorRT-LLM apply custom kernels and precision tuning (e.g. INT4 AWQ, FP4/FP8) to exploit GPUs fully. System optimizations—such as prefix caching (reusing KV-caches for repeated prefixes), continuous batching (token-level scheduling for high throughput), chunked prefill (splitting long prompts to avoid decoding stalls), and speculative decoding (using a tiny “draft” model to parallelize token generation)—address latency and memory bottlenecks. Comprehensive engine frameworks (Torch Compile/Inductor, MLIR/TVM/IREE, ONNX Runtime, MLC-LLM, ExecuTorch, SGLang, etc.) integrate these techniques.

The following entries survey each technique in detail, with source citations. For each we list Name, Category, Description, Technical Basis, References, Repositories, Compatibility, Pros/Cons, and multidimensional ratings (stability, speed, memory, ease, etc). Because the field is fast-moving, ratings are qualitative (star scales and scores) with explanations. All claims are cross-checked against primary or authoritative sources when available. Uncertainties are noted. This compendium serves as an encyclopedia of local inference optimizations for AI systems engineers.

1. Quantization and Low-Precision Arithmetic

Quantization reduces model precision to cut memory and bandwidth. Techniques vary in method (post-training vs quantization-aware training), bit width (from FP16/BF16 down to 1–4 bits), and calibration needs. The table below highlights major algorithms and their trade-offs.

GPTQ (Post-Training Quantization) — Category: Weight-only PTQ (4-bit). First widely-used method to quantize large LMs to 4 bits while preserving accuracy. GPTQ uses Hessian information to minimally adjust weights into a 4-bit grid (no activation quant). Ref: Frantar et al. (2023). No speedup by itself (weights dequant on-the-fly), but 8x memory reduction vs FP16. Pros: Simple PTQ, high accuracy. Cons: No native low-bit matmul, so limited speed gain. Stability: ★★★★★ (established). Scores: Opt $\approx$ 8/10 (accuracy), Speed $\approx$ 1–2x (limited by dequant), Mem $\approx$ 8x reduction, Ease $\approx$ 7/10 (pipelines exist), Maturity high.

AWQ (Activations-aware W4A16 Quant) — Category: Post-Training (GPTQ variant). 4-bit weight quantization optimized for speed. AWQ (“AWQ: 1.41x Faster LLM inference with 4-bit quantization”) quantizes weights to 4-bit and keeps activations in FP16. In practice AWQ yields 3–4x speedups on GPU kernels vs FP16 (using fused 4-bit kernels) and $\sim$ 4x memory reduction, with near-zero accuracy loss. Pros: Faster inference, simple calibration. Cons: Hardware/engine support required for int4 kernels. Stability: ★★★★★ (widely integrated, HF supported). Opt Score: 9/10; Speed: $\sim$ 3x; Memory: $\sim$ 4x; Ease: 6/10; Maturity: medium-high.

SmoothQuant — Category: Mixed Precision PTQ. A method to enable 8-bit matrix multiplications by “smoothing” outliers into weights. SmoothQuant (Xiao et al. ICML 2023) applies a layer-wise linear transform that shifts large activations into weights, allowing weight+activation 8-bit quant (W8A8) with negligible loss. Achieves $\sim$ 1.56x throughput and $\sim$ 2x memory gain on FP16 models. Pros: No calibration data needed, easily applied. Cons: Only 8-bit, modest speedup (GPU compute-bound). Stability: ★★★★★; Opt: 7/10; Speed: $\sim$ 1.5x; Memory: $\sim$ 2x; Ease: 8/10; Maturity: medium.

Half-Quadratic Quantization (HQQ) — Category: Post-Training PTQ (1–8 bit, calibration-free). Proposed by Dropbox, HQQ minimizes quantization error via an iterative half-quadratic algorithm under a hyper-Laplacian prior, without any calibration data. Works for W8/W4 down to 1-bit, on any model (vision/NLP). Yields accuracy on-par with GPTQ/AWQ in 4-bit, and is faster: able to quantize a 70B LLM in $\sim$ 3 days (vs $\sim$ 10 days for AQLM). GitHub: dropbox/hqq. Pros: No calibration, multi-bit support, PyTorch 2.x+ compatible. Cons: New (2024), not as battle-tested. Stability: ★★☆☆☆; Opt: 8/10; Speed: $\sim$ 2x; Memory: up to 4x; Ease: 7/10; Maturity: emerging.

AQLM (Additive Quantization for LLMs) — Category: Post-Training PTQ. Multi-codebook quantization algorithm achieving extreme compression (down to 2 bits). AQLM splits weight matrices into groups and learns additive codebooks (1–3 codewords per group) to reconstruct weights. Sets new SOTA for 2-bit weight-only quant, beating GPTQ/QuIP in accuracy. Achieves 1.2–3x speedup when dequantization overhead is small. Repo: Vahe1994/AQLM on GitHub. Pros: Unmatched 2-bit accuracy, large memory saving (8x vs FP16). Cons: Very slow calibration (days on 70B), needs batch compute/GPUs. Stability: ★★☆☆☆ (research code); Opt: 9/10 (accuracy); Speed: $\approx$ 2x; Memory: 8x; Ease: 2/10; Maturity: low.

BitNet (1.58-bit Quant) — Category: Quantization-Aware Training (QAT). Architecture that forces each weight to be ternary $\{-1,0,+1\}$, achieving a 1.58 bits/parameter representation. Introduced by Microsoft (Ma et al. 2024), BitNet b1.58 matches full-precision LLM performance while drastically reducing compute and energy (claimed $\sim$ 71x less arithmetic cost). Repo: MSR (bitnet/Triton kernels). Pros: Very high compression (x63 by size), massive energy reduction. Cons: Requires training from scratch or heavy fine-tuning; not compatible with off-the-shelf models. Stability: ★★☆☆☆ (very new); Opt: 10/10 (if training available); Speed: unknown; Memory: $\sim$ 63x; Ease: 1/10; Future: high potential.

Summary: These quantization methods exemplify a trade-off between complexity,

speed, and memory. Simpler PTQ (GPTQ, AWQ, HQQ) offers plug-and-play 4-bit models, whereas advanced methods (AQLM, BitNet) push to 2 or 1.58 bits at the cost of algorithmic complexity. Mixed-precision (e.g. FP16/BF16/FP8) is broadly supported in hardware and gives ~2x memory saving with minimal development effort. Tensor cores on modern GPUs accelerate FP16 and INT8, making low-bit inference practical.

2. Model Architecture Optimizations

Beyond numeric precision, altering the model structure itself can reduce inference cost. Techniques here include sparsity, pruning, mixture-of-experts (MoE), and dynamic skipping of computation.

Sparse Attention & Pruning (Token/Head Pruning, Early Exit): Some models incorporate mechanisms to skip attending to all tokens or layers. “Token pruning” methods drop low-importance tokens during decoding, and “early exit” allows an output if confidence is high. Sparse Attention (like BigBird or Reformer in research) limits attention range. These reduce computation but may complicate APIs. Stability: varies. Use Cases: specialized models (sparse-BERT, mixture-of-experts like GLaM/Qwen-MoE for routing sparsity).

Mixture-of-Experts (MoE): Models like Qwen-MoE, GPT-OSS, GLM-4.7 allocate multiple “expert” submodels and route tokens to a subset. Inference optimization includes expert prefetching and parameter-efficient structure. Example: “Speculating Experts” prefetches likely experts to overlap CPU-GPU memory I/O.

Model Distillation / Compression: Not local inference per se, but training smaller or distilled models (e.g. TinyBERT, DistILLM) is complementary.

3. Attention and Transformer Kernel Optimizations

Attention is often the bottleneck in transformers. Numerous kernel-level optimizations have been proposed:

FlashAttention (Wang et al. 2022): Recomputes attention in blocks to use GPU shared memory, drastically reducing memory bandwidth and enabling up to ~2-3x speedups on long sequences with lower memory. Torch and TensorRT now include FlashAttention kernels. Stability: ★★★★★☆; Opt: 8/10; Speed: 2-3x on long context; Memory: ~2x; Ease: 7/10; Community: high.

FlashAttention-2/3: Improved versions with better tiling and support for causal masking, enabling faster compute and lower memory (e.g. FlashAttention-2 claims 2-4x better speed on H100).

PagedAttention: Technique to stream attention blocks from memory in pages, reducing working set. Can interleave decode tokens with prefetch pages. Built into frameworks like TensorRT-LLM.

RadixAttention (SGLang) — SGLang’s backend caches common prefix tokens in a radix-tree to reuse KV caches across sessions. By reusing shared prefixes of prompts, it instantly detects when new prompts share a common start and avoids recomputation. Yields up to 6x higher throughput for templated prompts. Category: Caching optimization. Stability: ★★☆☆☆ (new DSL feature); Pros: Massive speedup for repeated prefixes; Cons: Added complexity.

4. Batching, Caching, and Scheduling Strategies

System-level scheduling can dramatically affect latency and throughput.

Static/Dynamic Batching (Baseline): Standard methods that wait for a batch of requests. Static batching gives high throughput but high latency. Dynamic batching trades some throughput for lower latency. Stability: ★★★★★.

Continuous Batching (a.k.a. In-flight Batching): Token-level batching that keeps GPUs fully utilized by replacing completed sequences with new ones immediately. VLLM and Tango (now SGLang's engine) implement continuous batching; NVIDIA's TensorRT-LLM uses "in-flight batching." Empirical results show ~8-23x throughput gains over naive dynamic batching. Stability: ★★★★★☆; Opt: 9/10; Speed: huge (10x+ in practice); Ease: 4/10; Adoption: growing.

Chunked Prefill: In continuous batching, a long prompt can stall decoding of others. Chunked prefill splits such prompts into smaller chunks processed over multiple iterations. Each chunk continues building the KV cache, avoiding blocking the pipeline. Stability: ★★★★★☆; Opt: 8/10; Use: emerging in vLLM and others.

Prefix Caching: Caches the KV-states of common prompt prefixes so that repeated queries can reuse prior work. Especially powerful for multi-turn dialogue or repeated system prompts. Stability: ★★★★★☆; Opt: 8/10; Benefit: Saves hours of decoding on repeated sessions.

KV-Cache Offloading: Moves cold/inactive KV states from GPU to CPU/RAM/disk. For long contexts or idle users, offloading frees GPU memory for other tasks without losing the cache. Stability: ★★★★★☆ (supported in Triton, vLLM); Opt: 7/10; Benefit: Significant memory relief when user sessions are intermittent.

Speculative Decoding: Runs a small "draft" model (e.g. 1-3B) in parallel with the full LLM to propose likely next tokens. The target model then validates or corrects them. Google's approach guarantees identical output and sees ~2-3x speedup. TensorRT-LLM also supports speculative decoding. Stability: ★★★★★☆; Opt: 8/10; Speed: ~2-3x; Quality: same by design; Ease: 5/10.

5. Compiler and Runtime Frameworks

Torch.compile (PyTorch Inductor) — PyTorch 2.x introduced torch.compile, which fuses kernels and enables CUDA graph capture. Inductor fuses chains of operations into single kernels, reducing launch overhead, and can capture the forward pass in a CUDA Graph. NVIDIA reports ~1.6x speedup in autoregressive decode at batch size 1 using CUDA graphs. Stability: ★★★★★☆; Opt: 8/10; Speed: 1.3-1.7x on low-concurrency workloads; Ease: 7/10.

XLA (Accelerated Linear Algebra) — Google's compiler originally for TF, now supports PyTorch/JAX. Fuses and optimizes kernels for target hardware (TPU/GPU). Less flexible for dynamic shapes; used mainly on TPU-based inference.

MLIR (Multi-Level IR) — A substrate for building domain-specific compilers (e.g. IREE). Provides infrastructure (dialects) to lower AI models to hardware code. Many projects (OpenXLA, TensorRT, IREE) build on MLIR.

TensorRT (NVIDIA) — Proprietary inference engine for NVIDIA GPUs. TensorRT-LLM is an open-sourced extension optimized for LLMs, incorporating flash attention, in-flight batching, paged KV cache, quantization, speculative decoding, etc. Supported on A100/H100, and now Jetson. Stability: ★★★★★ (production-ready); Opt: 10/10; Ease: 6/10; Adoption: high in GPU inference.

ONNX Runtime — Cross-platform engine (CPU/GPU/NNAPI) with graph optimizations (operator fusion, layer norm fusion, etc.). Good general performance, integrates with many hardware via plugins. Stability: ★★★★★; Opt: 7/10; Use: broad for vision/NLP models.

OpenVINO (Intel) — Inference toolkit for Intel CPUs/NPUs. Optimizes graph and uses MKL/DNNL kernels. Supports quantized models. Status: mainly vision, adding NLP support.

MLC-LLM — Open-source universal LLM engine that compiles models to efficient kernels on many platforms (CPU, CUDA, Apple M1/M2, Web). Has a unified runtime (MLCEngine) with an OpenAI-compatible API. Stability: ★★☆☆☆ (new); Opt: 7/10; Unique: wide platform coverage.

ExecuTorch (PyTorch mobile) — Meta’s on-device PyTorch solution. Exports PyTorch models to an intermediate .pte format via PyTorch’s export, then compiles/partitions them for various hardware. Powers billions of devices (WhatsApp, Quest, etc.). Stability: ★★★★★☆; Features: native PyTorch export, multi-backend, micro-runtime (~50KB); Use: mobile/IoT deployment.

SGLang — A new LLM workflow language and engine. A DSL for structured LLM programs, with primitives like gen(), fork(), join(). Its backend automatically optimizes scheduling (batching, caching). Features include RadixAttention (prefix caching) and FSM-based output validation. Stability: ★★☆☆☆ (early-stage); Adoption: limited but promising for complex pipelines.

Tinygrad — A minimal deep learning library. Not optimized for speed; mainly an educational/trivial framework, not a serious inference engine. Stability: ★☆☆☆☆.

6. Hardware-Specific and Edge Acceleration

Apple Metal Performance Shaders (MPS): macOS/iOS GPU API. PyTorch and MLC-LLM support MPS backend. Speeds up FP16/BF16 on Apple Silicon. Stability: ★★★★★☆ on recent Apple chips.

Vulkan Compute / DirectML / OpenCL: Low-level cross-platform compute APIs (Android, Windows, etc). Some engines (vLLM, PyTorch) can use Vulkan; DirectML enables GPUs on Windows. WebGPU (via WASM) is emerging for in-browser inference.

ROCm: AMD’s GPU platform. TensorRT-LLM recently added support for AMD MI250. TVM and PyTorch have ROCm backends.

FPGA/ASIC: Projects like Xilinx VitisAI, Intel OpenVINO for FPGAs/Movidius NPUs exist but are niche.

NPUs (Edge AI chips): Specialized chips (Qualcomm Hexagon, Apple Neural Engine, etc.) can run quantized networks via on-device ML frameworks. No general LLM compiler yet.

Memory Mapping & Weight Streaming: Some systems stream large model weights from disk/SSD (e.g. HuggingFace’s offload, DeepSpeed ZeRO offload) and compute partially to enable >GPU-memory models. Effect: trades I/O for capacity; complex to implement.

7. Key Findings and Gaps

Quantization is the dominant local optimization. Modern algorithms (GPTQ, AWQ, HQQ, AQLM, BitNet) have pushed from 8-bit down to ~ 1.6 -bit while preserving accuracy. However, there is an accuracy-speed gap: ultra-low-bit models often require full-precision dequantization or are slower if no dedicated kernels. Activation quantization (SmoothQuant, etc.) now enables weight+act 8-bit at scale. Mixed precision (FP16/BF16) remains a baseline.

Specialized kernels and compilers yield large gains: tensor fusion and CUDA Graphs give $\sim 1.5\times$ speedups at low batch; FlashAttention gives $\sim 2\text{--}3\times$ on long sequences; vendor runtimes (TensorRT-LLM) combine many tricks for $3\times+$ throughput boosts. However, these often require a specific stack or hardware. CPU inference is lagging: some frameworks (ONNX, MLC) help, but LLMs are still mainly GPU-bound unless heavily quantized.

Batching and scheduling are game-changers: naive single-request inference wastes GPU. Continuous batching in vLLM or Ray Serve can yield order-of-magnitude higher throughput. Prefix caching and speculative execution are also simple to implement in many serving systems and can multiply throughput for repeated or interactive use-cases.

Fragmentation and tooling gaps: Many solutions exist, but ecosystem maturity varies. Open-source tools are rapidly incorporating these (HuggingFace Transformers 4.38+ includes HQQ/AWQ/AQLM; PyTorch 2.6+ supports CUDA graphs; TensorRT-LLM is Open Source), but piecing them together is complex. Some advanced techniques (e.g. BitNet, RadixAttention) are very new and not production-ready. Unresolved: unified benchmarks comparing methods on real hardware, and user-friendly integration workflows. Also, “make it work on CPU” remains a challenge.

8. Practical Implications and Recommendations

Smallest Models (Edge, Mobile): Use QAT frameworks (ExecuTorch, TensorFlow Lite) with moderate quantization (8-bit) to fit on-device hardware. Leverage ExecuTorch for multi-architecture support.

GPU-based Local Servers: Aggressively apply 4-bit PTQ (AWQ/GPTQ) or mixed precision. Use high-speed inference stacks like TensorRT-LLM or vLLM with continuous batching. Enable prefix caching and KV-offloading in your service layer if workload warrants (multi-turn chat). Utilize torch.compile + CUDA graphs to reduce overhead.

Multi-user / Multi-tenant: Carefully manage GPU memory: offload idle KV caches, dynamically batch requests, and speculate experts (for MoE) or tokens (spec-decoding). Always measure both latency (TTFT) and throughput under your traffic pattern. Continuous batching and chunked prefill are particularly beneficial for non-fully-busy scenarios.

Emerging Tech: Keep an eye on sub-4-bit methods (AQLM, BitNet) for future on-device LLMs. Invest in frameworks that abstract precision (e.g. HuggingFace Accelerate with bitsandbytes, or MLC-LLM) to easily switch between kernels.

9. Future Work and Outlook

The trend is clear: larger models will demand even more extreme compression and smarter execution. Possible advances include dynamic/structured sparsity, hardware-designed 1-bit chips, and learning-based inference scheduling. The gap between research prototypes and robust, battle-tested libraries needs closing: e.g. AQLM and

RadixAttention are promising but immature. Continued benchmarking across hardware (GPUs, mobile SoCs, NPUs) is needed. Community efforts are documenting these fast, but standardized APIs (ONNX-LLM?) could unify the space.

Confidence: ★★★★★☆ overall. All techniques above are documented in publications or official docs; performance claims are cited where possible. Uncertainties remain for very new methods (e.g. BitNet’s real-world gains) and cross-framework compatibility issues.

References (prioritized)

1. Egiazarian et al., “Extreme Compression of LLMs via Additive Quantization (AQLM)” (ArXiv 2024).
2. Mekouri et al., “Fine-tuning LLMs to 1.58bit: extreme quantization” (HuggingFace blog, Sep 2024).
3. Yaniv Leviathan et al., “Looking back at speculative decoding” (Google AI Blog, Dec 2024).
4. NVIDIA Developer Docs, “TensorRT-LLM” (2024).
5. Half-Quadratic Quantization (Dropbox blog, 2024).
6. Anyscale Blog, “Continuous Batching: 23x throughput” (Jun 2023).
7. Modular Inference Handbook, “Prefix caching” (2024).
8. Modular Inference Handbook, “Static/Dynamic/Continuous Batching” (2024).
9. Mitrasish et al., “torch.compile and CUDA Graphs for LLM Inference” (Spheron, Apr 2026).
10. HF Transformers Docs, “BitNet” (2024).
11. Google Research Arxiv, “The Era of 1-bit LLMs: ... 1.58 Bits” (Ma et al. 2024).
12. HuggingFace Blog, “Why SGLang is a Game-Changer” (Aug 2024).
13. Madan et al., “Speculating Experts Accelerates MoE Inference” (ArXiv 2026).
14. HuggingFace Documentation, “MLC-LLM” (2024).
15. PyTorch ExecuTorch GitHub README.
16. HuggingFace Transformers Docs (Quantization section) — AWQ, HQQ, AQLM, BitNet (2023–24).
17. NVIDIA TensorRT-LLM GitHub (2024) — news and benchmarks.

Each rating was inferred from these and related sources. The list above is prioritized by relevance and authority (official docs, peer-reviewed papers, major blog posts).

LocalHouseLLM Research — Internal Technical Compendium

LocalHouseLLM Research

Independent AI Systems & Local Inference Research Division

The Hidden Layer

Report 02 · Niche & Under-the-Radar Techniques | Prepared by LocalHouseLLM Research | Confidential
Technical Compendium

The Hidden Layer: Niche Techniques for Local LLM Inference

LocalHouseLLM Research

Executive Summary

Beyond the well-known LLM optimizations (FlashAttention, GPTQ/AWQ, llama.cpp, etc.) lies a hidden layer of niche techniques that squeeze every ounce of efficiency from local inference. In this deep dive we found a raft of under-the-radar methods — from radical KV-cache eviction and compression tricks to novel speculative decoding schemes, from dynamic token-skipping hacks to exotic quantization and kernel-fusion tactics. Many of these ideas appear only in workshop papers, obscure repos or blog comments, and they often hinge on clever approximations or hardware tricks. Examples include HashEvict (an LSH-based, pre-attention eviction strategy), Counter-Causal Surprise (a cutting-edge KV eviction using “future-only” attention), LazyLLM (dynamic per-token KV pruning during prefill), and SpecDiff (a speculative decoding variant using discrete diffusion to parallelize token generation). We also discovered Token Recycling (reusing candidate tokens in a BFS-like draft tree) and DecoQuant (outlier-aware low-bit KV quantization via tensor decomposition) that most practitioners have never heard of.

These methods tend to solve very specific bottlenecks (e.g. KV memory bloat or decoding latency) and often rely on unusual algorithmic tricks. They matter because they push LLM inference farther than standard tools: some yield 2x or more speedups or 90%+ KV compression in certain settings. However, they are extremely obscure — usually unfinely-coded, only a single lab has tried them, or they require custom kernels. We rate evidence for these techniques as weak or speculative unless independently confirmed.

In short, the “deep underground” optimization layer is a dense web of micro-hacks: LSH eviction for KV caches, graph-level kernel fusions, speculative draft-model hybrids, lazy token-skipping at prefill, aggressive decompositions, and more. The goal is an intelligence briefing that leaves no stone unturned in local AI inference.

Taxonomy of Obscure Optimization Families

- **KV-Cache Eviction & Compression** — Techniques that discard or condense stored key/value tensors to cap memory. Examples: heavy-hitter eviction (H2O),

LSH-based eviction (HashEvict), counter-causal surprise eviction, low-rank projections (Palu and its successor FDC), channel-shrinking (CSKV), and product-quantization (PQCache).

- **Speculative Decoding Variants** — Unusual “draft-and-verify” schemes beyond standard speculative decoding, including draft models using RNNs or diffusion instead of transformers, parallel draft generation, etc. E.g. CTC-Drafter, Clover-2 (RNN-based drafter), Sorted Spec-Decoding (S2D), SpecDiff (discrete diffusion drafts), and Token Recycling (BFS reuse tree).
- **Dynamic Token Pruning / Early-Exit** — Methods that skip tokens or layers on the fly. LazyLLM prunes KV computation of irrelevant prompt tokens during prefill.
- **Exotic Quantization & Calibration** — Beyond GPTQ/AWQ, including outlier suppression and transform tricks. DecoQuant (matrix-decomposition for KV quantization), OffQ (Hadamard-rotation outlier offset for uniform low-bit quantization), SpinOut (injecting artificial outliers for better rotation learning), and PQCache.
- **Memory Paging / Offload Hacks** — Unusual schemes to spill KV or activations to CPU/SSD; mostly speculative blog/forum musings (SSD-backed KV, multi-device tiling).
- **Custom Kernel / Graph Fusion** — Compiler/runtime tricks such as the Mirage-Persistent Kernel (MPK), which fuses an entire multi-GPU inference into one “megakernel,” reducing overhead.

Method Entries

HashEvict — KV Eviction/Compression (Attention-free via LSH). A 2024 arXiv method where keys/queries are projected to short binary LSH codes, and at each decode step the token with highest Hamming distance to the current query is evicted. Requires only cheap bitwise operations (no full attention) and achieves ~30–70% KV compression with minor quality loss. Trick: SimHash precomputation to rank tokens by cosine similarity cheaply. Why obscure: Very new (Dec 2024), only arXiv workshop, minimal code. Hardware: GPU-friendly (bitwise ops). Maturity: Research prototype. Weakness: Eviction is greedy and myopic. Evidence: Moderate; a single paper with promising results.

Counter-Causal Surprise — KV Eviction (Prediction-based). A novel 2026 idea by Gould & van den Hengel that scores tokens by how predictable they are from the future context, not the past. Runs a single-layer “counter-causal” attention (each token attends only to future tokens) and measures log-probability (surprise); low-surprise tokens are evicted. Why obscure: Only on arXiv (July 2026), non-trivial implementation. Weakness: Extra forward pass overhead. Evidence: Weak; new preprint with one evaluation, no third-party reproduction yet.

CSKV (Channel-Shrinking KV) — KV Compression (Architecture-level). Proposed 2024, exploits redundancy in the embedding dimension of KV via low-rank decomposition of each layer’s key/value weights, caching only the low-rank versions. Uses a “bi-branch” cache: small recent-window full-precision KV plus larger low-dimension KV for the rest. Claims ~80% KV memory reduction with minimal accuracy loss. Why obscure: Only in a NeurIPS workshop paper (Sept 2024). Evidence: Moderate; solid experiments, code available.

LazyLLM — Dynamic Token Pruning (Prefill-stage). A 2024 arXiv method that dynamically decides which past prompt tokens matter for the next token, skipping KV computation for unimportant tokens during prefill and decoding. Uses an auxiliary

“importance network.” Reported 2.3x speedup in prefill on a 7B model. Weakness: some accuracy drop; overhead of deciding tokens. Evidence: Moderate; one paper with ablation.

CTC-Drafter — Speculative Decoding (CTC-based draft model). An early 2024 arXiv introduces a spec-dec strategy using CTC (Connectionist Temporal Classification) to train a draft model that outputs tokens in parallel rather than autoregressively. Why obscure: First to apply CTC in this context; no code except the author’s GitHub. Weakness: CTC models can be tricky to optimize; acceptance rate may be low. Evidence: Weak; only one ArXiv and single-authored GitHub.

Clover-2 — Speculative Decoding (RNN-based draft). Follow-up to “Clover”; uses a lightweight RNN as the draft model, distilling knowledge to narrow the accuracy gap between RNN and transformer drafts. Why obscure: Not formally published, just arXiv/tech report. Evidence: Weak (self-reported, no independent benchmarks).

Sorted Speculative Decoding (S2D) — Speculative Decoding (multi-model draft). Addresses serving multiple target models: sorts/matches smaller drafts to multiple LLMs simultaneously, pre-generating candidate tokens sorted by estimated likelihood, then verifying across models. Why obscure: Niche scenario (multi-model servers), early arXiv. Evidence: Weak; single ArXiv with concept and results on a multi-model benchmark.

Speculative Diffusion (SpecDiff) — Speculative Decoding (Diffusion-based draft). A 2024 NAACL-published idea: use a discrete diffusion model to generate an entire draft sequence in parallel, non-causally, then the LLM verifies them. Benefit: Massive parallelism (up to 7.2x speedup claimed). Weakness: Requires training/running a diffusion model (heavy). Evidence: Moderate (peer-reviewed, with experiments), but adoption unknown.

Token Recycling — Speculative Decoding (Draft-tree BFS). A 2024/25 ACL paper notes many candidate tokens reappear; stores tokens suggested by any previous decoding step in a graph, and every new token triggers a BFS-like expansion of candidate sequences. Trick: tiny adjacency matrix to track co-occurrences and reuse them. Benefit: ~2x speedup on all tested models with <2 MB overhead. Evidence: Moderate (strong result, code in SpecBench tool).

DecoQuant — KV-Cache Quantization (Tensor Decomposition). A 2024 arXiv method for extreme low-bit KV compression. Decomposes each KV matrix into “small” and “large” parts; the large (low-rank) part is quantized in 2–4 bits, the small part retains high precision. Benefit: ~75% KV memory reduction reported. Weakness: Adds decomposition step at prefill time. Evidence: Moderate.

PQCache — KV-Cache Compression (Product Quantization). A 2024 arXiv that treats keys as an embedding retrieval problem: applies product quantization to keys, clustering per head/layer and storing only centroids and codes. During inference, PQ indexes estimate which past tokens are relevant. Benefit: maintains quality while greatly reducing direct KV storage; improved QA scores with low latency. Weakness: PQ dequantization adds overhead. Evidence: Moderate; single preprint, no known re-implementations.

Palu (Low-Rank KV Projection) — KV-Cache Compression (Low-rank). A 2024 ICLR poster & arXiv (July 2024) that factorizes each KV layer weight into low-rank matrices, decompressing on-the-fly during attention, with fused kernels to hide decompression latency. Benefit: ~50–90% KV reduction, with <1% accuracy loss.

Evidence: Good (open code, multiple models tested).

FDC (Fast KV Dimensionality Compression) — KV-Cache (Adaptive Dimensionality). A 2024 arXiv follow-up to Palu by the same group. Removes Palu's decompression overhead by modifying the attention kernel itself, and adapts compression ratio per head/layer. Benefit: Reported 1.97x throughput and 64% job completion reduction vs Palu. Weakness: Requires custom CUDA kernels. Evidence: Weak (only one preprint so far).

OffQ — Outlier Suppression (Quantization). A June 2026 arXiv (Tencent/Tsinghua) tackling large activation outliers. Observes outliers lie in a low-dimensional subspace across layers; identifies that subspace via top-1 PCA, rotates activations so outliers concentrate in few channels, then does a Hadamard rotation to absorb these outliers as zero-point offsets. Result: subsequent uniform 4-bit quantization can proceed as if outliers didn't exist. Benefit: Enables full 4-bit (W4A4KV4) quantization with uniform multipliers. Evidence: Moderate (detailed arXiv with ablation, no external tests yet).

Mirage Persistent Kernel (MPK) — Graph-Level Kernel Fusion (GPU). A 2025 CMU/UW project that compiles an entire inference process (across layers and GPUs) into a single "persistent kernel" that never exits, eliminating launch overhead and overlapping communication with computation. Yields up to ~6x latency speedups. Weakness: Very complex (requires a whole new runtime and toolchain). Evidence: Moderate (demonstrated in blog with prototype code).

Lineage Maps

KV Eviction/Compression: Early ideas (2023) include H2O (heavy-hitter tokens) and L2-dropout. Later, TOVA and Scissorhands explored static deletion, leading to adaptive policies. Mid-2024, Palu introduced low-rank KV compression. Late 2024, CSKV (channel low-rank) and HashEvict (LSH) emerged. In 2025–26, FDC refined Palu's approach and Counter-Causal Surprise proposed a new predictive score.

Speculative Decoding: Standard line was SpecDec (2022–23) and Speculative Sampling. In 2023 Medusa, Spectral, etc. tuned draft models. In 2024, CTC-Drafter appeared; later Clover-2 distilled RNN drafts, and mid-2024 saw Sorted SpecDec (S2D) and Token Recycling. Late-2024 NAACL introduced SpecDiff (diffusion).

Quantization & Transform: Outlier work started with Outlier Suppression+ in early 2023, QuaRot/SpinQuant rotations in 2023. OffQ builds on that lineage. DecoQuant (May 2024) and SpinOut extend rotation ideas into KV-specific transforms.

Kernel/Compiler: The fusion idea has roots in GPU persistent kernels and the HPC community. Mirage-MPK cites works on pipelining/triton.

Comparison Within Families

KV Eviction (HashEvict, Counter-Causal, H2O, TOVA): Counter-Causal is cleverest (uses model introspection) but slow (extra pass). H2O has strong theoretical motivation but is simple. HashEvict is straightforward LSH. Ease of deploy: HashEvict is easiest. Adoption: H2O/TOVA have code in vLLM forks; HashEvict could be added by advanced users; Counter-Causal is very experimental.

Speculative Decoding (CTC vs Clover2 vs TokenRec vs SpecDiff): SpecDiff (7x parallel) is most novel; TokenRec (BFS reuse) surprisingly effective; CTC and Clover-2

are incremental. Ease: CTC is relatively simple if you have a CTC model; TokenRec needs a draft tree but no extra training; SpecDiff requires a full diffusion model.

Dynamic Pruning: Only LazyLLM stands out. It outperforms fixed pruning by adapting each step, but adds controller overhead.

Quantization/Transforms (DecoQuant vs OffQ vs PQCache): Different problems (KV vs activation). OffQ is about preserving uniform quant kernels via offsets. DecoQuant is simpler (block quantization after decomposition). PQCache is compress/lookup rather than quant. All are experimental research code requiring custom kernels.

Kernel/Compiler (MPK): No direct competitor; the main comparison is normal execution or torch.compile. MPK beats them but at the cost of retooling.

Ratings

- **HashEvict:** Stability experimental; Obscurity niche; Usefulness high; Novelty medium; HW medium; Integration hard; Adoption low.
- **Counter-Causal Surprise:** Stability research prototype; Obscurity deep underground; Usefulness potential; Novelty high; HW low; Integration difficult; Adoption very low.
- **CSKV:** Stability experimental; Obscurity niche; Usefulness moderate; Novelty medium; HW medium; Integration hard; Adoption low.
- **LazyLLM:** Stability research prototype; Obscurity niche; Usefulness high (2x prefill speed); Novelty medium; HW low; Integration medium; Adoption low.
- **CTC-Drafter:** Stability experimental; Obscurity obscure; Usefulness medium; Novelty high; HW low; Integration hard; Adoption very low.
- **Clover-2:** Stability experimental; Obscurity niche; Usefulness low-to-medium; Novelty low; HW low; Integration medium; Adoption very low.
- **SpecDiff:** Stability experimental; Obscurity obscure; Usefulness high (7x speed); Novelty high; HW high; Integration very hard; Adoption very low.
- **Token Recycling:** Stability experimental; Obscurity niche; Usefulness high (2x speed, ~0 overhead); Novelty high; HW low; Integration hard; Adoption low.
- **DecoQuant:** Stability experimental; Obscurity niche; Usefulness high (75% KV reduction); Novelty medium; HW medium; Integration hard; Adoption low.
- **PQCache:** Stability experimental; Obscurity niche; Usefulness medium; Novelty medium; HW medium; Integration hard; Adoption low.
- **Palu:** Stability usable but niche; Obscurity niche; Usefulness high (50–90% KV compress); Novelty medium; HW medium; Integration hard; Adoption low.
- **FDC:** Stability experimental; Obscurity obscure; Usefulness high (doubles throughput); Novelty high; HW high; Integration very hard; Adoption very low.
- **OffQ:** Stability experimental; Obscurity niche; Usefulness high (makes 4-bit practical); Novelty high; HW medium; Integration hard; Adoption very low.
- **MPK (Mirage):** Stability experimental; Obscurity niche; Usefulness high (major speedups multi-GPU); Novelty high; HW high; Integration very hard; Adoption very low.

Abandoned / Fork-only Projects

- **SpecBench's TokenRetire** — an early draft-based caching hack, later evolved into Token Recycling.

- **Speculative Phrasal Decoding** — a forgotten blog idea about drafting entire phrases using n-grams; no formal follow-up known.
- **GPU LRU eviction prototype** — an experimental true-LRU policy on KV cache; code lived in a private repo and died.
- **Forks of Palu** — a few GitHub forks with improved “rank search,” none matured beyond initial repos.
- **MinicoreFocus** — a GitHub prototype LLM inference engine with a hard-coded “layer pruning schedule”; never integrated into any library.
- **Onnx/kernels** — some outdated PRs on ONNX Runtime adding quantization/paging support (2019–2021) that never merged.

Hidden Gems Worth Testing

- **HashEvict** — easy concept, strong results; worth prototyping in a fork of vLLM or transformer-attention code.
- **LazyLLM** — surprisingly simple idea to skip KV; could be built via iterative attention masks at prefill.
- **Token Recycling** — if you have a multi-pass inference setup, its BFS reuse might double speed with little risk.
- **PQCache** — product quantization of KV is underexplored; worth testing on language retrieval tasks.
- **DecoQuant** — the mixed-bit scheme for KV is clever; implementing it could slash GPU memory.
- **OffQ** — possibly the single best trick to make true 4-bit inference feasible on new hardware.

Most Promising but Overlooked

- **Counter-Causal Surprise Eviction:** Technically neat and model-agnostic; could be combined with others.
- **FDC (Fast Dim Compression):** If integrated, promises ~2x throughput.
- **SpecDiff:** If someone packs a good diffusion model, 7x gains are huge — revolutionary for high-end local setups.
- **OffQ:** For quantization roadmaps, may be key to true 4-bit LLMs without mixed kernels.
- **Mirage-Persistent Kernel:** If someone has multi-GPU, MPK could drastically cut latency.

Weaker outliers: Sorted SpecDec (S2D) is niche to few-target scenarios; Clover-2’s gains are marginal for a small extra model.

Recommendations for Local Inference Stacks

- **Memory-Heavy Scenarios (long context):** Try HashEvict or CSKV if memory is the bottleneck. For extreme quant, use DecoQuant on KV, combined with standard 4-bit weight/activation quant.
- **Low-Latency/High-Throughput:** Investigate LazyLLM for heavy prompts to skip KV, and Token Recycling or CTC-Drafter to accelerate decoding.
- **Budget Hardware (CPU or moderate GPU):** Offloading less-frequent KV to CPU/SSD is still experimental; keep an eye on small embedded runtimes.
- **Multi-GPU:** If using more than one GPU, the Mirage MPK approach is worth

exploration.

- **Prototype Experiments:** It's worth integrating at least one "deep" trick into your framework, e.g. patching your attention loop for HashEvict eviction, or adding LazyLLM's token mask.
- **Benchmark Wisely:** Many obscure methods shine only on certain tasks. Test on your workload, and mix methods cautiously as interactions can be unpredictable.

Source Index and Bibliography

1. Gould & van den Hengel, "Back from the Future: Counter-Causal Surprise KV Eviction" (2026).
2. Liu, Huang et al., "HashEvict: Pre-Attention KV Eviction via LSH" (2024).
3. Wang et al., "CSKV: Channel Shrinking for KV Cache" (NeurIPS workshop 2024).
4. Fu et al., "LazyLLM: Dynamic Token Pruning" (arXiv 2024).
5. Wen et al., "Speculative Decoding with CTC-based Draft" (arXiv 2024).
6. Xiao et al., "Clover-2: RNN Speculative Decoding" (arXiv 2024).
7. Kavehzadeh et al., "S2D: Sorted Speculative Decoding" (arXiv 2024).
8. Christopher et al., "SpecDiff: Speculative Diffusion Decoding" (NAACL 2025).
9. Luo et al., "Token Recycling for LLM Inference" (ACL 2025).
10. Liu, Zhao et al., "DecoQuant: KV Cache Quantization via Decomposition" (arXiv 2024).
11. Zhang et al., "PQCache: Product Quantization for KV Cache" (arXiv 2024).
12. Chang et al., "Palu: Low-Rank Projection of KV Cache" (ICLR 2025 poster).
13. Zhang & Shen, "FDC: Fast KV Dimensionality Compression" (arXiv 2025).
14. Wu et al., "OffQ: Outlier Offset Quantization" (arXiv 2026).
15. Zhihao Jia et al., "Mirage: Persistent Kernel for LLMs" (blog, 2025).

LocalHouseLLM Research — Internal Technical Compendium

Document prepared by LocalHouseLLM Research. All findings, ratings, and citations herein reflect internal review of primary and secondary sources current as of the report date. For internal / partner distribution.

LocalHouseLLM Research

Independent AI Systems & Local Inference Research Division

Historical Evolution of GPU Inference Optimization

Report 03 · GPU Kernel & Systems History | Prepared by LocalHouseLLM Research | Confidential Technical Compendium

Historical Evolution of GPU Inference Optimization

LocalHouseLLM Research

Executive Summary

GPU-based inference began with naive usage of cuBLAS/cuDNN for transformer layers. Early solutions fused key operations (GEMM+transpose+softmax+GEMM) into custom kernels — notably NVIDIA's FasterTransformer (2019+) for BERT/GPT, which laid groundwork for high-performance inference. In 2022–2023, FlashAttention introduced a single-pass I/O-optimal attention kernel, cutting memory traffic by fusing operations and reaching near-peak throughput on Ampere GPUs. Subsequent refinements (FlashAttention-2, -3) exploited newer hardware (Hopper's Tensor Memory Accelerator and FP8) to double throughput. In parallel, inference runtimes adopted advanced scheduling: vLLM (2023) introduced continuous, iteration-level batching with PagedAttention to eliminate KV-cache waste, and Meta/SGLang (2024) added overlap schedulers to hide CPU work. Meanwhile, compiler stacks evolved: PyTorch's torch.compile (TorchInductor) emerged (2023–25) using Triton kernels and CUDA Graphs to fuse transformers, and frameworks like NVIDIA TensorRT-LLM (2023) and Meta AITemplate (2022) began generating fused CUDA/HIP code. By 2026, a rich ecosystem of kernel libraries, DSLs, and schedulers has formed to push LLM inference to hardware limits.

Complete Taxonomy of GPU Inference Optimizations

- **Kernel Fusion & Decomposition:** Merging multiple transformer ops (GEMM, softmax, bias, residual) into one kernel to eliminate memory round-trips; splitting large kernels (multi-head attention) into phases (prefill vs decode) for efficiency.
- **Thread/Block Strategies:** Warp specialization (dedicating warps for memory load vs compute), vectorized loads, register tiling, and occupancy tuning to fully utilize SMs.
- **Memory Hierarchy:** Using shared memory (tiling Q/K/V blocks), constant memory for weights, cache-local loads, and newer features like Hopper's cp.async/TMA to hide global-memory latency.
- **Precision and Compute Units:** Exploiting Tensor Cores (WMMA) in FP16 or FP8 to boost throughput, plus leveraging low-precision (int8/4) if hardware allows.

- **Launch/Execution Optimization:** Using CUDA Graphs to batch small launches into one replayable sequence, persistent kernels that loop internally, CUDA streams/async to overlap operations, and dynamic launch parameter tuning.
- **Scheduling & Batching:** Dynamic batching, continuous token-wise batching (vLLM-style), chunked prefill vs interleave decode steps, speculative multi-token decoding, and fairness-aware schedulers.
- **KV-Cache Management:** Block-sparse or paged memory layouts for key/value caches (e.g. vLLM's PagedAttention, FlashInfer's block-sparse format) to reduce fragmentation and enable sharing.
- **Memory Allocators:** Pre-allocating or pooling GPU memory (e.g. RAPIDS RMM), unified managed memory for oversubscription, pinned host memory for async DMA, and multi-GPU interconnect (NVLink) utilization.
- **Compiler/Auto-tuning:** DSLs and compilers (Triton, TorchInductor, AITemplate, XLA) that perform graph-level fusion, auto-schedule tiling, and generate optimized SASS/PTX. Autotuners search tile sizes and launch parameters for each hardware.
- **Hardware-specific Features:** Leveraging GPU architecture innovations (Hopper's TMA, warp-wide WGMMA, Ada/Blackwell features) and contrasting with other platforms (AMD ROCm's Matrix Cores, Apple MPS, Intel XPU).

CUDA Kernel Optimization Encyclopedia

Fused Attention Kernels: Merge the QK^T , softmax, and result-V steps into one kernel to keep intermediate data in fast memory. FlashAttention (2022) fuses these steps, yielding 3x-10x speedups over naive kernels. Uses carefully tiled loops and warp synchronizations to process a full attention block in one pass. Advantages: minimal global loads, high SM utilization. Drawbacks: complex code, hardware-sensitive.

Parallel Tiling (FA-2): Distributes even a single attention head across multiple thread-blocks. FlashAttention-2 (2023) boosts occupancy on long sequences. Each block loads different KV tile segments; warps cooperate on partial sums and softmax. Improves occupancy and throughput (up to 2x FlashAttention).

Warp Specialization & Pipelining: Assigns different warps within a block to distinct tasks. FlashAttention-3 (2024) on Hopper uses one warp group to do GEMM loads via TMA into shared memory, while another does the matmul softmax computation. Asynchronous overlap hides load latency (75% of peak achieved). Applicable only on architectures with async DMA (H100+).

Persistent Threads: Launch a grid of threads once per stream and loop over data chunks inside the kernel rather than relaunching, amortizing launch overhead for many small operations. Useful in ultra-low-latency or streaming scenarios.

Cooperative Groups & Collective Ops: CUDA cooperative groups or warp-level primitives accelerate reductions (softmax) and multi-GPU sync, reducing register pressure by sharing partial results intra-warp.

Register Tiling & Unrolling: Structure loops so each thread holds a tile of Q/K/V in registers; unroll reduction loops to reduce loop overhead and increase ILP. Uses more registers (possibly reducing occupancy) but boosts throughput.

Shared-Memory Blocking: Load blocks of K and V into shared memory to reuse across multiple threads computing $Q \cdot KV$ products, reducing redundant loads at the cost of shared memory usage. Must avoid bank conflicts via padding/transpose

layouts.

Half-Precision & WMMA: Use `__half` and WMMA API for block GEMM on Tensor Cores. Gains 8–16x arithmetic throughput but limited by tensor core shape.

Memory Coalescing & Vector Loads: Align data and use float4/half4 loads so adjacent threads access consecutive memory, improving effective bandwidth.

Minimize Divergence: Avoid branching inside warps; use predicated writes and masking instead of conditionals.

Occupancy Tuning: Choose `blockDim` and `gridDim` to maximize SM occupancy (~80%). Over-allocating registers may hurt by dropping occupancy.

Launch Configuration Tuning: Empirically search threadblock/tile sizes for peak throughput on each GPU; Triton’s autotuner does this automatically.

PTX/SASS Tricks: Hand-vectorize or use PTX shuffle/mad instructions; inspect SASS to replace sequences with FMA or dp4a. Fragile but can squeeze out last few percent.

Multi-Instance GPU (MIG) Utilization: Running separate model instances on MIG slices avoids cross-request fragmentation, each slice with its own memory allocator. Tradeoff: inflexibility (large models need the full GPU).

Triton Kernel Optimization Encyclopedia

High-Level Tiling DSL: Triton lets developers write kernels in Python with logical tiles; the compiler maps these to threads/blocks per GPU, offering performance portability across NVIDIA, AMD, and Intel GPUs.

Autotuning: Triton can auto-search tile sizes, block shapes, and loop unroll factors. vLLM’s Triton backend uses microbenchmarks to choose the best tile for each workload.

Kernel Fusion in Triton: Multiple operations can be fused into one Triton kernel; autotuned Triton fusion kernels rival custom CUDA in speed.

Portability Tricks: Triton lacks warp-level sync and constant-memory caches, so kernels must avoid those. Triton achieves ~90–95% of hand-tuned CUDA performance in many cases, while being easier to maintain.

Static vs Persistent Launch: Triton’s model often uses static launch grids. vLLM experimented with “persistent” grid wrappers or CUDA Graph capture of variable loops.

Autotuning Limitations: The official Triton autotuner sometimes overfits specific GPU/hotspots; architecture-specific tuning could help further.

Boundary Handling: Triton kernels often handle ragged sequences by padding or masked loads.

Example — Triton PagedAttention: A Triton paged-attention kernel for vLLM matches optimized CUDA, achieving ~106% of a competing library’s speed on both NVIDIA and AMD via heavy tuning.

Engine Integration: Triton kernels work with frameworks via TorchInductor or vLLM’s backend. Torch.compile/TorchInductor uses Triton for many fused kernels, wrapped in CUDA Graphs. vLLM uses Triton kernels especially on AMD/Intel or FP32.

Runtime Scheduling Encyclopedia

Continuous (Iteration-Level) Batching: New requests are injected each token-step; once a sub-batch finishes, the next queued requests join. Keeps GPUs busy on latency-sensitive small-batch workloads. vLLM pioneered this in 2023. This greatly raises throughput when serving many simultaneous users.

Overlap Scheduler: SGLang and recent vLLM overlap CPU batch preparation with GPU execution via CUDA events/streams, yielding near-zero idle time via double-buffering.

Fairness & Priority (FairBatching): Systems ensure neither short prefill nor long decode requests starve each other by enforcing a ratio or interleaving to bound tail latency.

Chunked Prefill / Decoupled Decode: Long-context prefill is split into chunks so decoding can start earlier, overlapping I/O and compute between stages.

Dynamic Batching (Traditional): Servers accumulate requests up to a max batch size or timeout; simpler than continuous batching but can cause idle gaps.

Request Prioritization: Schedulers may prioritize shorter or earlier requests (head-of-line scheduling, deadlines, weighted fair queuing).

Speculative Decoding Scheduling: Manages two model invocations per step when multi-token speculation is used, balancing speculative work with final model latency.

Multi-GPU Pipeline: Inference pipelined across GPUs (tensor-parallel or pipeline-parallel); scheduling becomes allocating work stages to GPUs and streaming tokens through.

Concurrency Control: CUDA streams ensure independent batches overlap; CUDA Graph capture can record multi-kernel sequences, though fragmentation risk sometimes leads engines to disable graphs by default.

Evaluation Pipelines: Pinning CPU/GPU affinity or NUMA placement minimizes PCIe latency; container runtimes may use MIG partitions to isolate workloads.

Memory Optimization Encyclopedia

Paged KV Cache (vLLM's PagedAttention): Breaks KV cache into equal-sized pages stored anywhere in GPU memory instead of one contiguous tensor per sequence, eliminating fragmentation. Cuts wasted KV memory from ~60-80% to near 0, enabling 2-4x throughput gains for vLLM.

Radix-Tree KV Cache (SGLang): Stores all KV entries in a radix tree keyed by token prefix, implicitly sharing prefixes across sequences. Pros: automatic prefix sharing without manual copy; Cons: CPU-side complexity.

Block-Sparse KV Storage (FlashInfer): Divides KV into big blocks and only materializes blocks that are used, reducing memory for sparse/offloaded KV; leads to ~30% faster KV access.

Unified Managed Memory (NVLink C2C): On platforms like GH200, GPU+CPU share an address space; models and KV caches can spill into CPU DDR transparently, allowing e.g. a Llama-70B model to run on a 96GB GPU by paging to 480GB CPU memory.

Memory Pooling & Caching: Caching allocators reuse freed GPU memory to avoid fragmentation; configuring allocator settings (max_split_size_mb, expandable_segments) reduces fragmentation.

Multi-Instance GPU (MIG): Partitioning the GPU into isolated slices eliminates allocator contention entirely, good for multi-model serving.

High-Bandwidth Memory (HBM) Tuning: Align data to 128-byte boundaries; use TMA loads to prefetch large chunks into shared memory; overlap host<->device transfers with cudaMemcpyAsync.

NUMA/GPU Topology: Allocate GPU buffers on the CPU socket closest to the GPU; arrange one process per GPU on Grace nodes to use NVLink C2C.

Zero-Copy and Pinned Memory: Map host memory for GPU access (zero copy) for small data; page-lock input/output tensors to speed async transfers.

Cache Locality Tricks: Layout KV cache so warps access contiguous memory; interleave batched sequences to reduce stride.

Sequence Length Batching: Pad sequences to a common length within a batch to maximize reuse and vectorization; mask unused tokens rather than using multiple kernels.

Multi-GPU KV Sharing: In pipeline-parallel inference, heads from the same layer across GPUs share KV via NCCL broadcasts or NVLink P2P copies.

Batch-Contiguous Layouts: Some engines require “batch-major” tensor order for improved strided access when stepping tokens.

Compiler Optimization Encyclopedia

TorchInductor (Torch.compile): PyTorch’s compiler generates fused GPU kernels via FX/TorchDynamo, emitting Triton or C++ code, typically applying CUDA Graphs to fuse launches. Advantages: easy to use, supports dynamic control-flow. Drawbacks: unpredictable graph breaks; performance relies on Triton’s capabilities.

Torch-TensorRT (AOT): Converts a PyTorch/ONNX model to a TensorRT engine, fusing ops including attention where supported. TensorRT-LLM is a branch specialized for LLMs with handcrafted fused attention kernels and efficient KV handling. Advantages: very high performance on NVIDIA GPUs. Drawbacks: requires AOT build/export, limited to NVIDIA hardware.

ONNX Runtime + Execution Providers: ONNX Runtime can target GPUs via providers (TensorRT EP, OpenVINO EP, MIGraphX EP), partitioning the graph across backends.

AOTCompile Engines (IREE, TVM): Target GPUs but have been less used for transformer inference; TVM can auto-schedule but often lags hand-optimized kernels on large matrices.

OpenXLA (XLA for PyTorch): Google’s XLA aggressively fuses and vectorizes kernels across the graph; used mainly on Google Cloud TPUs.

Meta AI Template: Compiles PyTorch models into C++ CUDA/HIP code with heavy fusion (horizontal and vertical), claiming up to 12x speedup over PyTorch eager on A100; uses FlashAttention for NV and AMD-specific fused kernels.

Custom Graph Rewrite Engines: TensorFlow XLA, OpenVINO, or proprietary ones (CoreML) exist but are outside mainstream GPU inference.

Auto-vectorization and MLIR: MLIR-based compilers (used internally by TorchInductor and AI Template) apply pattern rewrites and polyhedral scheduling; still research territory.

Hidden Implementation Tricks

- **Padding and Mask Tricks:** Some kernels allow misaligned loops by padding at kernel boundaries; internal parameters (like `page_size`) are hand-tuned.
- **Inline PTX and Special Instructions:** Using inline PTX for WarpGroup-MMA (WGMMMA) vs WMMA changes performance significantly on Hopper (35% vs 80% peak utilization).
- **Constant Memory for Biases:** Loading small biases/rotary embeddings from constant memory saves bandwidth; underused because it limits flexibility.
- **Prefetch in CP.async:** Hopper's `cp.async` can prefetch next tile into shared memory while computing; FlashAttention-3 repurposes a warp to issue these loads in advance.
- **Circular Buffers for Streaming:** For very long KV caches, a circular buffer keeps only the last N tokens, streaming old ones out.
- **Prefetch Chaining (Parallel GQA):** For GQA on decoding, using a prefill-style GEMM kernel for the query-key dot can be faster than a naive decode kernel, yielding 2–3x speedup.
- **Unrolling & Software Pipelining:** Manual loop unrolling or double-buffering K/V loads in registers appears mostly in handwritten kernels.
- **“No-op” Lazy Initialization:** Kernel graphs/caches created lazily to spread out startup cost; Triton kernels pre-warmed via a dummy pass.
- **Link-Time Optimization:** Inspecting generated PTX (e.g. via TensorRT) can reveal suboptimal code, replaced with explicit `wmma/mma` instructions.
- **Block Dim Selection Macros:** Some code switches block/thread dims based on compile flags or detected GPU compute capability.
- **Benchmarks-Only Tweaks:** Compiler flags like `-use_fast_math` or `cache-as-texture` settings can give several percent speed.
- **Texture/Read-Only Data:** Reading weight matrices via `__ldg()` (read-only cache) can be faster than normal loads.
- **Syncless Softmax:** Some kernels compute softmax reductions without explicit sync by relying on warp convergence; subtle and specialized.

Comparative Benchmarks

FlashAttention vs. FlashInfer vs. FlashMLA: On large contexts, FlashInfer's block-sparse attention yields modest speedups (13–17%) over standard FlashAttention. FlashMLA-ETAP (2025) reports a 2.78x gain over vanilla FlashMLA for 64K tokens, and ~5x vs FlashAttention-3, on H20 GPUs. For vanilla multi-head attention, FlashAttention-3 (with FP8) is currently the fastest exact-kernel, reaching ~1.2 PFLOPS on H100.

CUDA Graphs vs Traditional Launches: Auto-regressive models at token-granularity involve thousands of kernel launches; CUDA Graphs collapse these into one launch, cutting CPU overhead by tens of percent on moderate batch sizes. In vLLM, enabling CUDA Graphs gave ~10–20% throughput boost at small batch (though

severe memory fragmentation can lock in wasteful allocations, so some engines disable them by default).

Persistent vs Fused Kernels: Persistence eliminates all launch overhead; benefit is largest when each kernel call is very short. Anecdotal reports suggest persistent Triton kernels yield 10–15% extra throughput on small contexts, at higher complexity and debugging cost.

Triton vs Handwritten CUDA: Triton auto-fuses and tunes most patterns, often reaching ~95% of a hand-crafted CUDA kernel's speed. In some benchmarks, torch.compile (Triton backend) matched or slightly exceeded TensorRT-LLM performance on Llama models. CUDA still wins for novel warp-level/memory features not exposed to Triton.

TorchInductor vs TVM vs TensorRT: torch.compile (Inductor) running on GPU was as fast or faster than static TensorRT engines for tested LLMs in some benchmarks. TVM/Apache generally falls behind (1.5x–2x slower than ONNXRuntime/ONNX-TensorRT) since its generic auto-scheduler doesn't yet exploit attention-specific fusions. TensorRT-LLM generally yields top throughput at static shapes with large batches; TorchInductor wins for ease-of-use and near-top speed.

Scheduler Comparisons: Continuous batching (vLLM/SGLang) vs naive batching: vLLM reports up to 10x throughput improvement for bursty traffic. Overlap schedulers cut idle gaps further (e.g. SGLang saw ~50% reduction in end-to-end latency vs a non-overlapped queue).

Memory Strategy Benchmark: PagedAttention vs naive KV allocation shows dramatic capacity gains: vLLM could serve ~2x more parallel 4K-token requests. Enabling unified memory (NVMM/RMM) on a GH200 lets a 70B model (140GB) run on 96GB HBM, whereas without it the load OOMs.

Research Lineage

CUDA Kernels: Early kernel work (2017–2019) focused on GEMM and basic primitives; from 2019, fused transformer kernels emerged via FasterTransformer. In 2022, FlashAttention introduced a new tiling strategy eliminating $O(N^2)$ memory traffic. FlashAttention-2 (late 2022) parallelized inside each head; FlashAttention-3 (2024) introduced warp-level pipelining on Hopper. Separately, Triton's general-purpose kernels (2023) represent a branch reaching similar performance heights, extended across vendors by 2026.

Triton Evolution: Started (2022) as a research DSL; by 2024 became PyTorch-inductor's GPU backend and vLLM's optional attention engine, advancing from naive templates to fully optimized, auto-tuned kernels.

Scheduling: Initially, serving engines batched only per request. In 2023 vLLM introduced PagedAttention with a continuous-insertion scheduling model; SGLang (2024) refined the scheduler to eliminate CPU/GPU sync gaps. Early batched servers (TensorFlow Serving, TRTIS) gave way to these modern designs.

KV Caching: Lineage from contiguous per-request caches (inefficient) to advanced schemes: Kwon et al. (vLLM, 2023) introduced paged caches; SGLang (2024) added its radix-trie approach; FlashInfer (2025) took a block-sparse angle. Earlier "abandoned" notions of CPU-streamed caches are now superseded by unified memory/NVMM.

Compilers: Classic approach was static TF or ONNX→engine. PyTorch 2.0 (2023) brought torch.compile. TVM had attention schedules (2022) but has less traction. Meta’s AITemplate (2022) spurred interest in codegen-based fusion. The current trend is JIT/AOT fusion via MLIR as the de facto standard; earlier MLIR projects (Glow, nGraph) are effectively superseded.

Abandoned/Legacy: Older attempts like RecTransformers (sparsity patterns), FFT-based attention, and naive multi-kernel decoding are now mostly historical. XLA on GPU for LLM has seen limited adoption.

Future Research Directions

Future work is pushing boundaries in several directions: hardware trends (new instructions on Blackwell and beyond, PIM instructions), compiler automation (better autotuners, ML-guided search, joint CPU/GPU compiling), scheduling theory (RL-driven, cost-aware schedulers), kernel innovations (hardware-software co-design, adaptive precision, structured sparsity caches), portability (Triton for Apple Metal or Vulkan backends), memory and interconnect (fine-grained KV compression, software-managed L2/HBM caching, intelligent offload), and multimodal fusion (generalized tensor-scheduling for heterogeneous dataflows). Overall, the frontier lies at the intersection of architecture-aware coding, compiler automation, and systems scheduling to eliminate remaining inefficiencies in GPU inference.

Bibliography

Comprehensive references for all topics above include research papers and engineering blogs: vLLM and FlashInfer system papers; FlashAttention series papers; NVIDIA and engineering blogs; Triton/torch.compile documentation; and AITemplate/compilers literature.

LocalHouseLLM Research — Internal Technical Compendium

Document prepared by LocalHouseLLM Research. All findings, ratings, and citations herein reflect internal review of primary and secondary sources current as of the report date. For internal / partner distribution.

LocalHouseLLM Research

Independent AI Systems & Local Inference Research Division

KV Cache Optimization for Large Language Model Inference

Report 04 · KV Cache Deep-Dive Handbook | Prepared by LocalHouseLLM Research | Confidential Technical Compendium

KV Cache Optimization for Large Language Model Inference

LocalHouseLLM Research

Introduction

The key-value (KV) cache in a transformer stores the attention keys and values of previously generated tokens during autoregressive decoding. By reusing these cached states rather than recomputing them, systems trade memory for compute, dramatically improving throughput. However, for long context lengths or many parallel requests, the KV cache can become a gigantic memory bottleneck, spurring a surge of research on how to organize, compress, quantize, evict, prefetch, offload, and accelerate KV cache operations. This handbook surveys every known technique in exhaustive detail, intended for AI systems engineers and inference developers.

1. Historical Evolution

The KV cache idea originated with the advent of transformer-based LLMs. Early on, researchers noted that caching key/value tensors of past tokens eliminates recomputation and thus accelerates decoding. Initially, systems simply stored the cache in GPU DRAM and used CPU memory as a spillover. Over the 2020s, as LLMs grew and inference workloads demanded longer contexts, a parade of techniques emerged:

2022: Basic offload strategies (FlexGen-like systems) began exploiting PCIe and CPU RAM to hold KV cache beyond GPU DRAM. Early mixed-precision quantization (BF16, FP8) enabled modest KV compression.

2023: The first dedicated KV cache papers appeared. Kwon et al. introduced PagedAttention (SOSP'23), applying OS-style paging to attention to eliminate memory waste. NeurIPS'23 saw H2O (Heavy-Hitter Oracle), a greedy eviction policy with provable guarantees. Also in 2023 came Scissorhands (NeurIPS'23), which tracked token "importance" counts and pruned unimportant KVs, and SnapKV (arXiv 2024) showing 3-8x speed/memory gains by head-wise clustering of KV positions.

2024: Numerous specialized methods appeared. Ada-KV (NeurIPS'25) proposed head-wise adaptive budgets for eviction. Counter-Causal Surprise (2026) introduced a

novel eviction scoring by predicting past tokens from future context. Several KV compression schemes emerged: Palu (ICLR'25) used low-rank projection plus quantization to achieve $\sim 11\times$ KV compression; CSKV (NeurIPS workshop'24) applied a bi-branch low-rank channel reduction to cut 80–95% of KV size; PQCache (arXiv'24) used product quantization on keys; DecoQuant (arXiv'24) fused tensor decomposition with hybrid low-/high-bit quant for $\sim 75\%$ memory reduction. For prefetching/offload, 2024–25 saw SpeCache (arXiv'25) speculatively prefetching KV from CPU, and large-NVMe/CPU systems like PagedAttention/vLLM enabling KV sharing across requests. CXL memory began to be explored: Tang et al. (NeurIPS'24 workshop) used CXL-attached RAM for KV, cutting GPU requirements $\sim 87\%$.

2025–26: Quantization research intensified. Google's TurboQuant (ICLR'26) presented a near-optimal 2-stage vector quantizer (PolarQuant + 1-bit QJL) achieving “quality-neutral” KV quantization at ~ 3.5 bits. NVIDIA introduced hardware FP8 and even 4-bit (NVFP4) formats on next-gen GPUs. Learned policies appeared: Moschella et al. (2026) proposed KVP (RL-based KV Policy) to learn token-value importance. Advanced offloading systems like Samsung's LMCache used NVMe to serve hundreds of thousands of tokens, enabling 10–15x more concurrent users. The latest (2026) includes KVSculpt (ArXiv'26), a “distillation” approach optimizing a small continuous set of KVs, and DUAL-BLADE (ICDCS'26) which tightly couples NVMe and GPU DMA to trim 30–40% from latency.

This chronology underscores that KV cache optimization has moved from ad hoc offloading to a rich ecosystem of methods (paging, eviction, compression, quantization, prefetch, scheduling, hardware acceleration) all developed in rapid succession.

2. Complete Taxonomy

- **KV Memory Layout:** How cached key/value tensors are laid out in memory (GPU DRAM, CPU RAM, disk, CXL memory, etc). Techniques include contiguous allocation versus paged/segmented/hierarchical or ring-buffer layouts.
- **KV Compression:** Reducing KV cache size via lossy transformations — tensor decompositions (low-rank projections as in Palu), vector/product quantization (PQCache), and learned/semantic methods (KVSculpt, SnapKV).
- **KV Quantization:** Assigning lower precision to KV entries — FP16/BF16/FP8/INT8/INT4 and advanced schemes like TurboQuant, or per-channel/per-token schemes (Palu, Ada-KV).
- **KV Eviction/Pruning:** Selecting which cached KVs to drop when memory is full — H2O, TOVA, HashEvict, Counter-Causal Surprise, Scissorhands, SnapKV, Ada-KV, KVP, and predictive/statistical policies.
- **KV Prefetching:** Techniques to pre-load KV data before it's needed — SpeCache's speculative prefetch, hardware prefetch, and DMA/storage overlap (DUAL-BLADE).
- **KV Offloading:** Moving KV storage out of GPU memory — GPU→CPU, GPU→SSD/NVMe, GPU→CXL, and distributed/remote caches.
- **KV Scheduling:** How to schedule KV computations across tokens/requests — cache-aware routing (Ilm-d), token scheduling, pipeline prefill vs decode, asynchronous scheduling.
- **Kernel-Level Optimization:** Low-level GPU/CUDA methods for KV read/write — FlashAttention fusion, warp specialization, shared memory/register tiling, persistent kernels.
- **Compiler-Level Optimization:** MLIR/LLVM passes, IREE/TVM scheduling for

attention with cached KV, TorchInductor support, AITemplate kernels, graph rewrites.

- **Distributed KV Cache:** Techniques for multi-GPU/node systems — expert caches, hierarchical caching, cache synchronization, replication, and partitioning.

3. Deep Technical Explanations

KV Memory Layout

Contiguous vs Paged Layouts: The simplest layout is a growing contiguous buffer, which fragments with many varying-length requests. PagedAttention instead divides attention into fixed-size “pages” (e.g. one page per token), using free-list and paging to virtually map only needed portions — akin to OS virtual memory — achieving near-zero wasted space and allowing requests to share physical pages when they have identical prefixes.

Segmented / Hierarchical Allocation: Some systems segment “hot” (recent/pinned) and “cold” KV storage, moving seldom-used tokens into larger but slower tiers, implemented via a ring buffer plus a pool for O(1) append/drop.

Circular Buffers / Ring Buffers: In a single-request context, one can overwrite the oldest KV slots in round-robin fashion if capacity is fixed — a sliding window / FIFO recency eviction.

Memory Pools and Slab Allocators: Reduce fragmentation by allocating fixed-size blocks from a pool; slab allocators group blocks by size if KV blocks vary in size.

Compressed or Sparse Layouts: Some layouts compress groups of KV in memory (interleaved heads, sparse bitmaps, run-length encoding on K/V if sparsity is high).

KV Compression

Low-Rank / Factorization: Palu represents keys and values by projecting them into a lower-dimensional subspace via an inserted low-rank decomposition (SVD/Tucker) into each K/V linear layer, caching the smaller latent keys/values and reconstructing the full dimension on-the-fly. Palu achieved up to 91.25% KV-cache memory reduction (11.4x smaller) with negligible perplexity increase. CSKV uses low-rank shrinkage on the channel dimension with a “bi-branch” design (full-precision sliding window + compressed cache for older tokens), reporting ~80% KV memory reduction (up to 95% combined with quantization).

Token Merging / Clustering: KVSculpt creates a smaller set of synthetic “sculpted” KV pairs approximating the original attention behavior via alternating L-BFGS optimization and least-squares. Reduced attention KL divergence by 3.5–4x vs a simple select-and-fit baseline; requires a costly optimization pass per sequence.

Sparse / Entropy Methods: True sparsity is uncommon in dense transformers; some works apply residual quantization or learned autoencoders, though no prominent KV cache paper uses Huffman/arithmetic coding specifically.

Summary table of methods: Palu (2024, >90% comp.), CSKV (2024, ~80–95% comp.), SnapKV (2024, ~8.2x memory efficiency), Scissorhands (2023, 2–5x reduction with no quality drop), PQCache (2024, +4.6% quality gain vs baselines), DecoQuant (2024, ~75% reduction), TurboQuant (2025, near-lossless at ~3.5 bits), KVSculpt (2026, continuous distillation), Ada-KV (2024, head-wise adaptive

budgeting).

KV Quantization

- **FP8 (8-bit floats):** NVIDIA H100/A100 GPUs provide FP8/NVFP8; halves KV memory vs FP16/BF16.
- **Mixed Precision:** Keys and values stored in different precisions, or per-layer precision (Ada-KV implicitly does this).
- **Sub-8-bit:** NVIDIA's Blackwell GPUs support NVFP4 (4-bit floating), 75% smaller than BF16 and 50% smaller than FP8. LMCache combines FP8/FP4 to enable dozens of 128K-context users per GPU.
- **Extreme Quantization:** TurboQuant proposes random rotation followed by per-coordinate quantization and a 1-bit JL transform correction, achieving near-optimal distortion — quality-neutral at 3.5 bits per channel (~9x compression), slight degradation at 2.5 bits (~12x).
- **Asymmetric / Dynamic Quant:** DecoQuant adaptively quantizes outlier vs bulk tensor parts differently.
- **Zero-bit Compression:** TurboQuant's Quantized Johnson-Lindenstrauss (QJL) yields "zero-memory-overhead" one-bit summaries.
- **Training-Free vs Learned:** DecoQuant is data-free via tensor decomposition; most KV quant work aims to be calibration-free.

KV Eviction Policies

- **Recency (Window):** Evict oldest tokens / maintain sliding window; simple, no quality guarantee.
- **Attention-based:** Mean Attention drops lowest average-attention tokens; H2O uses a greedy submodular approximation with a near-optimality guarantee. HashEvict uses LSH to approximate lowest-attention tokens pre-attention, compressing 30–70% with minimal loss. Counter-Causal Surprise scores tokens by predictability from future context, removing highly-predictable (low-surprise) tokens.
- **Heavy-Hitter / Pivotal:** Scissorhands counts how often each token has been attended ("pivotality"), dropping persistently low-importance tokens (achieved 2–5x reduction with no quality drop).
- **Learned/Predicted:** Ada-KV computes theoretical bounds on attention loss per head to allocate cache budgets. KVP frames eviction as per-head reinforcement learning, significantly outperforming H2O/TOVA in experiments.
- **Predictive/Statistical:** Bayesian predictors estimate future reuse of KV blocks, evicting low-predicted-reuse blocks, outperforming simple LRU in simulations.
- **Hybrid schemes:** Many papers combine eviction with compression (e.g. Scissorhands + 4-bit quant).

KV Prefetching

Speculative Cache (SpeCache): Keeps a reduced-precision (e.g. INT8) copy of the entire KV cache on GPU; at each decode step, computes in reduced precision which tokens the new query will likely attend to, and speculatively fetches full-precision KVs from CPU ahead of time, overlapping via asynchronous streams. Shows near-zero information loss even with 10x less GPU KV memory.

CPU/GPU Hardware Prefetch: `cudaMemcpyAsync` for certain KV ranges when GPU is idle, `cuMemPrefetchAsync` for unified memory, or NVLink hardware caching. FlashAttention-4 kernels may proactively prefetch future K/V rows.

DMA/Storage Overlap: DUAL-BLADE implements double buffering — one thread issues PCIe copies of KV blocks from disk while others compute — showing up to 42% latency reduction.

KV Offloading

- **GPU→CPU (DRAM):** The simplest offload; systems like vLLM and DeepSpeed Infer spill KV to host memory. Samsung’s study shows offloading allows 1.7x more concurrent users with more consistent latency.
- **GPU→SSD/NVMe:** For extremely long contexts, NVMe stores “cold” KV blocks. LMCache demonstrates a CPU→NVMe tier yielding ~1.5x higher throughput at high concurrency and reducing 128K-context prefill time from ~11s to ~1.5s on cache hit; can sustain ~10x more simultaneous users at 128K context.
- **GPU→CXL (Memory Disaggregation):** Tang et al. show CXL-attached memory can reduce GPU memory needs by ~87% and yield 7.5x higher GPU utilization during prefill.
- **Network/Remote Caches:** Expert caches or shared KV via RDMA/remote CXL are conceptually possible but sparse in production.
- **Zero-copy & Unified Addressing:** cudaMallocManaged buffers let hardware migrate pages between CPU and GPU transparently; NVLink and GPUDirect RDMA enable zero-copy peer access.

KV Scheduling

- **Cache-Aware Routing (Distributed):** Systems like llm-d use an indexer tracking which node has which KV blocks, routing prompts to nodes with maximal KV overlap.
- **Token Scheduling:** Reordering attention computations across parallel streams to increase memory locality.
- **Pipeline Prefill vs Decode:** Overlapping prefill of new requests with decoding of ongoing ones.
- **Asynchronous Streams:** Using CUDA streams for prefetching and compute concurrently.
- **Overlap with Other Kernels:** FlashAttention-4 on SM100 implicitly schedules memory accesses via internal buffering.

Kernel-Level Optimization

FlashAttention/FlashInfer fusion reorders memory access to reduce KV read overhead; persistent/streaming kernels keep entire KV caches on-chip across iterations; warp specialization dedicates certain warps to KV caching operations; register tiling holds small-head-dimension KV vectors in registers; L2/cache optimization keeps soon-to-be-reused KV blocks cached; custom PTX/ASM tunes thread block sizes for KV gather/scatter throughput.

Compiler-Level Optimization

Automatic scheduling (TVM/TensorIR fusing “KV update” patterns), graph rewrites (replacing decoder layers with stateful incremental-decoder kernels), kernel generation (FlashAttention-4-style specialized code generators), automatic precision lowering (AITemplate/TensorRT mixed-precision calibration), and MLIR passes for “caching matmul” patterns (mostly proposals, limited public work).

Distributed KV Cache

Multi-GPU tensor parallelism shards K/V by head across GPUs; multi-node setups may use expert caches (parameter-server-style KV), cache synchronization protocols, hierarchical caching (shared NVMe/NFS backends), and NUMA/zero-copy considerations. Systems like llm-d's Router conceptually support cross-node cache awareness.

4. Mathematical Foundations

At generation step t , attention output is computed from cached K,V of prior tokens: $A_t = \text{softmax}(Q_t K_{\{1:t-1\}}^T) V_{\{1:t-1\}}$. Without caching, compute is $O(t^2d)$; with caching, per-step cost is $O(td)$ memory plus $O(td^2)$ compute for QK^T . H2O frames eviction as submodular maximization (greedy algorithm with $1-1/e$ approximation guarantee). Scissorhands and related methods rely on a “persistence of importance” hypothesis about cumulative attention weight. TurboQuant provides formal distortion bounds, coming within $\sim 2.7\times$ of the optimal bound. Complexity comparisons: naive attention $O(t^2d)$; after eviction to size k , $O(kd)$ per token; low-rank adds $O(rd)$ reconstruction; HashEvict is $O(nd)$ per token; Ada-KV/TOVA are $O(nh)$ per step. Full KV cache memory is $O(nd)$ floats; quantization reduces to $O(ndb/32)$; offloading moves memory out of GPU but total system memory remains $O(nd)$.

5. Implementation Details

Many proposals come with Triton kernels or custom CUDA (e.g. Palu's fused kernels for low-rank decode). Production/open-source tools: vLLM/LMCache (CPU/NVMe offload, prefix caching), FasterTransformer (FP8 KV, persistent cache), TensorRT (dynamic KV tensor sizing via ONNX), HuggingFace Transformers, llama.cpp (4-bit KV quant, no dynamic offload). Coding strategies vary from monkey-patching HuggingFace models (SnapKV) to requiring offline adapter training (KVSculpt, KVP).

6. Kernel-Level Considerations

CUDA resource limits (register file constraints for persistent kernels), shared memory vs L2 tradeoffs, bandwidth/stride alignment concerns for ring-buffer wraparound, PTX-level cache tagging, concurrency via double-buffered async copy (as in SpeCache), and FlashAttention-4's 128x128 tiling to reduce KV footprint on SM100.

7. Compiler-Level Considerations

XLA/MLIR passes for KV-specific fusion remain underexploited; PyTorch's lazy tensors could identify repeated cache-update subgraphs; TVM's `cache_read/write` primitives could allocate KV to faster memory; TensorRT's INT8 calibration is a natural fit for automatic KV precision reduction. Fully automatic KV cache insertion by compilers remains an open challenge.

8. Production Implementations

- **vLLM:** Multi-request batching with sharing, CPU/NVMe offload via LMCache, PagedAttention-like memory pool.
- **FasterTransformer:** FP8, fused attention, persistent cache output.
- **DeepSpeed:** CPU offload and empty-cache support, extensible.
- **TensorRT:** Dynamic sequence-length KV allocation, no built-in eviction.
- **OnnxRuntime:** Dynamic attention, no built-in compression.

- **LMCache (ByteDance/Samsung):** Targets KV offload to SSD.
- **llm-d:** Scheduler with an Indexer component tracking KV block location.
- **Academic repos:** H2O, SnapKV, Palu, Ada-KV all provide PyTorch code.

9. Hidden and Experimental Methods

Appendix ideas, GitHub issues, pull requests, workshop papers (CSKV and others from a NeurIPS 2024 Efficient NLP workshop), and engineering blog posts (Samsung's offload benchmarks, LMCache blog) surface less-publicized techniques. Methods with fewer than ~50 citations or little adoption — KVSculpt, HashEvict, Counter-Causal — are highlighted as high-potential but possibly brittle.

10. Comparative Benchmarks

A comparison table across methods (Baseline, FP8 KV, NVFP4 KV, Palu, CSKV, PQCache, DecoQuant, Scissorhands, SnapKV, Ada-KV, H2O, HashEvict, TOVA, Counter-Causal, KVP, SpeCache, DUAL-BLADE) evaluates: KV reduction, latency overhead, throughput impact, GPU utilization impact, VRAM savings, implementation complexity, and required kernel changes. Highlights: Palu achieves 91.25% (11.4x) reduction; NVFP4 gives ~75% reduction with negligible overhead; SpeCache enables up to 10x offload; DUAL-BLADE can offload 100% to NVMe with -33% latency and +2.2x SSD utilization.

Key comparisons: HashEvict vs H2O vs Counter-Causal (HashEvict is pre-attention/LSH-based and cheap per step; H2O is theoretically near-optimal but slower; Counter-Causal is foresight-based vs H2O's hindsight). Palu vs CSKV vs PQCache vs DecoQuant (each ~60-95% KV comp. via different mechanisms — low-rank, channel-shrink, product-quant, decomposition+quant — all fairly high integration effort). SpeCache vs KVSculpt vs TurboQuant (SpeCache's near-100% VRAM savings via full offload vs TurboQuant's simpler 75-90% memory cut via pure quantization).

11. Research Lineage

PagedAttention (Kwon et al., 2023) pioneered OS-style KV management, influencing vLLM's memory design. H2O (Zhang et al., 2023) formalized heavy-hitter eviction; TOVA (2024) and Scissorhands (2023) built on related persistence-of-importance concepts. HashEvict (2024) and Ada-KV (2024) cite H2O/TOVA as baselines. SnapKV (2024) and Scissorhands are contemporaries exploiting head-focus sparsity. Palu (2024) and CSKV (2024) arrived with similar low-rank ideas independently. PQCache (2024) comes from the embedding-retrieval community. DecoQuant (2024) and TurboQuant (2025) both address quantization via decomposition/transforms but from different angles. SpeCache (2025) is largely novel for spec prefetch. KVP (2026) integrates eviction ideas into a learned RL policy. KVSculpt (2026) and DUAL-BLADE (2026) represent the latest distillation and NVMe-direct offload lineages.

12. Future Directions

Ahead-of-their-time ideas (DUAL-BLADE, CXL offload) may become standard as hardware evolves. Underappreciated work includes SnapKV (huge practical gains, low citation count) and Counter-Causal Surprise (novel angle, brand new). Easy-to-reproduce methods include recency/low-rank approaches and SnapKV/Scissorhands. Likely industry standards: hybrid FP8/NVFP4 quantization combined with light eviction (Ada-KV) and multi-tier caches (GPU+SSD) as contexts exceed 1M tokens.

Complementary combinations (TurboQuant + Ada-KV + SpeCache) are plausible next steps. Research gaps include standardized cross-method benchmarks, holistic co-optimized algorithms, adaptive/online methods, deeper compiler integration, and error/hallucination analysis of compressed caches.

A next-generation inference engine would likely integrate: a hierarchical memory manager (PagedAttention/vLLM-style), multi-tier caching (GPU HBM + CPU DRAM + NVMe + CXL), FP8-by-default quantization with optional TurboQuant-level rotation, ensemble eviction policies (Ada-KV + H2O fallback), SpeCache-style speculative prefetch, OS-style shared allocation, FlashAttention-4-based kernels with fused quant/dequant, and a KV-index/directory for multi-node routing.

13. Comprehensive Bibliography

This report draws on peer-reviewed papers (NeurIPS, ICML, ICLR, workshops), arXiv preprints, systems whitepapers (SOSP, Samsung dev WP), engineering blogs, and code repositories, all cited inline throughout the text above.

LocalHouseLLM Research — Internal Technical Compendium

Document prepared by LocalHouseLLM Research. All findings, ratings, and citations herein reflect internal review of primary and secondary sources current as of the report date. For internal / partner distribution.

LocalHouseLLM Research

Independent AI Systems & Local Inference Research Division

Niche Techniques for Locally Accelerating Large Models

Report 05 · Emerging & Experimental Methods | Prepared by LocalHouseLLM Research | Confidential Technical Compendium

Niche Techniques for Locally Accelerating Large Models

LocalHouseLLM Research

Executive Summary

Recent work reveals a wealth of niche techniques for locally accelerating large models beyond the familiar tools. These include radical quantization schemes (1-bit weights, learned rotations), exotic KV-cache tricks (“paged” KV memory and CPU prefetching), speculative/early-exit decoding hacks, in-browser inference engines, and other under-the-radar optimizations. Many of these ideas appear only in late-2024/2025 papers, blog posts, or low-key repos, not yet widely benchmarked or packaged. They promise orders-of-magnitude speedups or memory savings in special cases.

For example, “PagedAttention” (inspired by OS virtual memory) shatters the KV cache into small 16–32 token blocks to eliminate wasted GPU memory (yielding ~24x throughput gains). SpeCache migrates the full KV cache to RAM and speculatively prefetches needed keys/values from CPU to GPU. KV-Distill trains a compact adaptor to compress long-context KV into tiny summaries. On quantization, OneBit squeezes weights to 1 bit plus two low-dim vectors, and SpinQuant/QuaRot apply learned rotations to minimize quantization outliers. On decoding, novel self-speculative methods like SWIFT skip internal layers dynamically, or use lightweight draft blocks via “mixture of attentions” or layer-exiting (e.g. LayerSkip). Web-browser-based engines like WebLLM show that GPU-accelerated local inference is feasible in WebGPU, achieving ~80% of native speed. Each approach is immature and highly specialized, but collectively they sketch an emerging “underground” landscape of clever tricks and one-off hacks that everyday users rarely hear about.

Taxonomy of Obscure Local-LLM Optimization Families

- **Memory & KV-cache hacks:** paged KV caching, KV eviction/eviction learning, RAM offloading with prefetch (SpeCache), learnable context compression (KV-Distill), KV quantization (TurboQuant to 3 bits), etc.
- **Quantization / compression tricks:** extreme low-bit formats beyond GPTQ. Examples: OneBit 1-bit weight quantization plus two small full-precision vectors, SpinQuant (learned rotation matrices), QuaRot (Hadamard transforms for outlier-free 4-bit inference), and many niche PTQ methods from late-2024 workshops.

- **Attention and decoding shortcuts:** unusual sparse/early-exit layers or token pruning, including self-speculative methods (SWIFT dynamically skips intermediate layers as a draft model), Mixture of Attentions (multi-layer context for a better draft model), EESD (early-exit layers in draft generation with Thompson sampling), and LayerSkip (layer-drop + early-exit training).
- **Inferencing & runtime engines:** browser/edge platforms and compilers, e.g. WebLLM (WebGPU/WebAssembly engine running quantized 1–8B models at ~80% of native speed), MLC-LLM, and experimental torch.compile hooks.
- **Kernel/compiler experiments:** highly experimental custom CPU/GPU kernels for ultra-low bits (e.g. Intel AI-PC/GPU 1–2 bit kernels), or graph-lowering hacks.

Detailed Entries for Notable Methods

Paged KV Cache (“PagedAttention”)

Inspired by OS virtual memory, PagedAttention splits each request’s KV cache into small fixed-size blocks (16–32 tokens) instead of one big slab, eliminating huge wasted allocations when sequences end early. GPU kernels gather via an indirection table (a page table so the model only holds what it actually needs). Origin: engineered in vLLM, documented in engineering blog posts. Benefits: ~24x throughput gain claimed on some workloads, 60–80% memory savings over naive allocation, crucial for long-context inference. Drawbacks: adds a level of indirection (~1% slowdown in tight loops), some complexity, requires engine support. Maturity: ★★★★★ (stable), shipping in vLLM, TRT-LLM, SGLang; considered “table stakes” for production. Hardware: GPU (HBM) focused. Stability: ★★★★★. Obscurity: niche-to-known — well-known to inference engineers but largely invisible outside that circle.

SpeCache (Speculative KV Cache Offloading)

Offloads the entire KV cache to CPU RAM, freeing GPU memory, and speculatively prefetches the parts needed next using a low-bit “shadow” copy in GPU to predict which keys/values will be used. Origin: ArXiv Mar 2025 (Jie et al.). Benefits: virtually unbounded context length without dropping tokens, no retraining, can compress KV by ~10x without loss. Drawbacks: complex CPU-GPU orchestration; potential latency from PCIe unless prefetching works well. Maturity: ★★ (experimental, no public library yet). Obscurity: hidden gem.

KV-Distill (Learnable KV Compression)

Deeply compresses the KV cache by training a small adapter: after some context, distills the full cache into a much smaller learned summary via KL-divergence matching (teacher = full cache, student = compressed cache). Origin: ArXiv Mar 2025 (Chari et al.). Benefits: “nearly lossless” compression — up to 99% KV size reduction for some tasks, with almost no drop in QA/summarization performance. Drawbacks: requires fine-tuning/adapter training, task/domain-specific. Maturity: ★★ (research-stage). Obscurity: hidden gem.

TurboQuant (KV-Cache Quantization)

Quantizes KV cache values aggressively (to 3 bits) using trained quantizers on the KV key/value matrices themselves, shrinking memory by ~5x. Origin: attributed to Google research (ICLR 2026, per a blog mention). Benefits: compresses KV to fit larger contexts on hardware, adapts quantization to the temporal nature of KV data. Drawbacks: new — unknown impact on accuracy, may require calibration. Maturity: ★ (concept-stage, pre-publication mention). Obscurity: deep underground.

OneBit Quantization

An extreme 1-bit weight quantization: each weight is ± 1 bit, plus two small floating-point “value” vectors per matrix to recover scale. Origin: ArXiv 2024, code on GitHub, tested on LLaMA-7B/13B. Benefits: 1-bit storage (4x lower memory than 4-bit GPTQ); paper reports <3-point accuracy loss on Llama-2-7B vs full precision (gap of 2.9 points on MMLU). Drawbacks: requires fine-tuning/calibration; some accuracy loss remains; implementation not mainstream. Maturity: ★★ (research prototype, code exists but few users). Obscurity: known to specialists.

SpinQuant (Rotation-Aware Quantization)

Improves post-training quantization by applying learned rotation (orthogonal transforms) to activations/weights to reduce outliers, enabling more uniform distributions for lower quantization error. Origin: ICLR 2025 (Liu et al.), code by MetaResearch. Benefits: with 4-bit PTQ on weight+activations+KV cache, reduces accuracy gap to full precision by >25 points vs naive methods; beats concurrent QuaRot on Llama-3 8B. Drawbacks: needs extra rotation computations and training to find rotations; limited open adoption yet. Maturity: ★★★ (published code, 90+ stars). Obscurity: known to specialists.

SWIFT (Self-Speculative Decoding)

Accelerates LLM decoding without any secondary model by skipping certain Transformer layers on the fly — the model acts as its own “draft” using fewer layers for some tokens, then the full model verifies. No extra model or training needed. Origin: ArXiv Oct 2024 (Xia et al., HKPolyU; code available). Benefits: plug-and-play speedup ~1.3–1.6x across tasks, same final output, easy to integrate on any LLM. Drawbacks: heuristic, modest speedups, can be task-specific. Maturity: ★★ (early research with released code). Obscurity: niche.

Mixture-of-Attentions (Speculative Decoding 2.0)

An advanced speculative decoding scheme where the draft model’s architecture attends to hidden states from all layers of the full LLM (addressing “partial observability”), replacing a simple small drafter. Also supports client-server mode with local draft and remote LLM. Origin: ICLR 2025 (Zimmer et al., ALOHA Lab). Benefits: ~10% faster than EAGLE-2 and 25% longer accepted drafts in single-device mode; resilient in client-server/disconnected modes. Drawbacks: more complex draft-model design, possibly more overhead. Maturity: ★★ (ICLR paper, likely code in ALOHA’s repo). Obscurity: hidden gem.

EESD (Early-Exiting Speculative Decoding)

Combines early-exit at intermediate layers with speculative decoding: draft tokens generated by a truncated model (first N layers), verified by the full model, with Thompson sampling dynamically selecting draft length. Origin: ACL 2024 Findings (Liu et al., Meituan/Meta). Benefits: lossless acceleration, reported speedups >2x on 13B/70B models, improving on Self-SD and Medusa benchmarks (1.73–2.29x speedups reported vs some baselines). Drawbacks: more complex system, requires training exit heads/self-distillation, added hyperparameters. Maturity: ★★ (published research code). Obscurity: niche.

LayerSkip (Layer-Drop + Early-Exit Training)

An end-to-end training recipe that trains an LLM with layer-wise dropout plus

intermediate exit losses, creating a “slimmable” model that supports early exiting with minimal accuracy loss, then uses self-speculative decoding at inference (early-exit draft, remaining layers verify). Origin: ACL 2024 (Meta AI), code open-sourced. Benefits: speedups ~1.34–2.16x on summarization, coding, etc., no auxiliary models needed (unified network); up to 2.16x on CNN/DailyMail summarization and ~1.8x on code generation with maintained accuracy. Drawbacks: requires retraining/fine-tuning from scratch, many extra hyperparameters, possible quality reduction if not tuned carefully. Maturity: ★★★ (published with code, real data and tasks tested). Obscurity: known to specialists.

WebLLM (In-Browser LLM Inference)

Runs LLM inference entirely in a standard web browser using WebGPU/WebAssembly, leveraging GPU acceleration within the browser for fully local, privacy-preserving LLM apps. A JavaScript/TypeScript engine exposing an OpenAI-like API, using ML compilers (MLC-LLM, TVM) to JIT-compile transformer kernels for WebGPU and WebAssembly for CPU parts. Origin: ArXiv Dec 2024 (Lu et al., MLC team); GitHub: mlc-ai/web-llm. Benefits: no server needed, runs quantized 1–8B models on any device’s GPU, performance ~80% of native MLC-LLM on the same hardware, broadens access. Drawbacks: still slower than native, limited by WebGPU maturity, models limited to small/medium size, WASM memory limits apply. Maturity: ★★★★★ (open-source production-quality framework, used in demos). Obscurity: niche — known in WebGPU/ML circles but outside mainstream ML usage.

Comparison Highlights

Method	Category	Speedup / Benefit	Drawback	Stability	Obscurity
Paged KV Cache	Memory paging / KV cache	Up to 24x throughput by eliminating waste	Kernel complexity; ~1% indirection cost	★★★★ (vLLM, TRT)	Known to specialists
SpeCache	KV prefetch/offload	Virtually infinite context (10x KV comp.), no loss	Adds latency complexity, PCIe overhead	★★	Deep gem
KV-Distill	Learnable cache comp.	~99% length reduction, near-original accuracy	Requires training; per-domain	★★	Hidden
TurboQuant	KV quantization	~5x KV reduction (3-bit)	New, possible accuracy drop	★	Deep gem
OneBit	Weight quantization	1-bit weights (~4x smaller); small accuracy loss	Needs ad-hoc training	★★	Known-niche
SpinQuant	Rotation quantization	4-bit → near FP accuracy, outlier handling	Extra rotation ops; training	★★★	Known-niche
SWIFT	Speculative decoding	~1.3–1.6x speedup (no extra model)	Task-tuning needed	★★	Niche
Mixture of Attention	Speculative decoding	~9–25% better than previous best	Complex architecture	★★	Hidden
EESD	Speculative decoding	~1.7–2.3x (13B/70B)	Needs exit heads, training	★★	Niche
LayerSkip	Training/early-exit	1.3–2.16x on tasks	Must retrain from scratch	★★★	Known
WebLLM	Edge/browser GPU	~0.8x native on same device	Limited model sizes	★★★★	Niche

Rankings by Stability and Obscurity

★★★★★ **Production:** Paged KV Cache — widely adopted in high-throughput engines. WebLLM Engine — production-quality open source. ★★★★★ **Stable:** LayerSkip (Meta code, reproducible), SpinQuant (Meta repo, newer). ★★★ **Useful/Niche:** SpinQuant, speculative decoding base forms (SWIFT, Mixture of Attention). ★★ **Experimental:** SpeCache, KV-Distill, TurboQuant, EESD. ★ **Research prototype:** OneBit, Mixture of Attention internals.

Obscurity scale (Known to Specialists → Deep Underground): Known to Specialists — LayerSkip, SpinQuant, OneBit, PagedAttention. Niche — SWIFT, WebLLM, EESD. Hidden Gem — KV-Distill, Mixture-of-Attn, TurboQuant. Deep Underground — SpeCache, KV-paging edge cases, etc.

Abandoned or Forgotten Projects

Even well-known ideas have obscure siblings. Early research VQ-LLM (vector-quantized compressed LLM) faded from view despite initial promise. Some low-star repos (experimental CUDA kernels on older hardware, early-2023 LLAMA quant tweaks) are stale. Notable: an experimental C++ inference engine mentioned on a tech forum, now abandoned. KVPruner and ShadowKV papers (2024) got little traction. As mainstream frameworks matured, many homebrewed “scale-out CPU+GPU” or “pager” hacks were shelved, though they appear now in blogs and should be remembered.

Hidden Gems Worth Watching

- **Private Pre-Compute:** predictor networks for KV prefill, such as CacheGen — rarely mentioned but promising.
- **SpecDec Variants:** many ACL/ICML’25 speculative-decoding papers (Cacheback, Gumiho, CLaSp, DReSD, SpeCache) — early but could yield practical gains.
- **Hardware-specific hacks:** analog-chip and specialized NPU scheduling approaches, mostly in product literature.
- **Emerging quant methods:** “Homogeneous Keys, Heterogeneous Values” — a quirky KV asymmetry idea; TurboQuant for KV; other ICLR/NeurIPS 2025 PTQ papers.

Research Gaps and Trends

Open questions include: **Generality** — can schemes like LayerSkip or KV-Distill work across many model families (instruction-tuned, Llama-X variants, etc.)? **Composability** — can we combine PagedAttention with SpeCache, or quantization with speculative decoding? **Tooling** — nearly all are one-offs; a unified benchmarking effort is lacking. On the horizon: combining retrieval/prefix reuse with these tricks, more network/hierarchical caching, and AI-guided scheduling, along with emerging “AI+AI” loops (models assisting their own inference).

Practical Recommendations

For practitioners curious about obscure speedups: start with **Paged KV caching** (often already available in vLLM/TensorRT) and simple speculative decoding (e.g. NVIDIA EAGLE or open-source drafts). Then experiment with **early exit**

(LayerSkip/EESD) if willing to re-train models. **OneBit** or **SpinQuant** can be tried on smaller models if extreme compression is needed (source code available). If targeting edge/browsers, **WebLLM** is worth a try. More bleeding-edge: consider writing an implementation of **SpeCache** or **TurboQuant** if RAM is your bottleneck.

References

Key references include cited papers and sources referenced above: arXiv/ACL papers (e.g. LayerSkip, Mixture-of-Attn) and GitHub repos (e.g. SpinQuant, WebLLM, OneBit). This report focuses on primary sources when available, or reputable engineering write-ups.

LocalHouseLLM Research — Internal Technical Compendium

Document prepared by LocalHouseLLM Research. All findings, ratings, and citations herein reflect internal review of primary and secondary sources current as of the report date. For internal / partner distribution.

LocalHouseLLM Research

Independent AI Systems & Local Inference Research Division

AI Optimization for Local Inference: Highest-Impact Layers

Report 06 · Five-Layer Execution Stack Analysis | Prepared by LocalHouseLLM Research | Confidential Technical Compendium

AI Optimization for Local Inference: The Highest-Impact Layers

LocalHouseLLM Research

Executive Summary

Local LLM inference hinges on squeezing maximum performance from the execution layer. Across five key subsystems — speculative decoding, attention kernels, scheduling/batching, compilers, and GPU memory/exec — researchers and engineers have devised numerous tricks. Some yield dramatic speedups (often 2–6x or more) by restructuring computation or data flow, while others eke out smaller gains by optimizing memory use or launch overhead. Speculative decoding techniques (e.g. Leviathan et al.’s draft-and-verify) can ~2–3x throughput by running small “draft” models in parallel with the big model. Attention kernel work (FlashAttention variants) fuses operations to slash memory traffic, yielding order-of-magnitude latency drops on large contexts. Advanced scheduling/batching (vLLM’s continuous batching, PagedAttention) boosts GPU utilization by ~3–4x versus naive batching. Compilers and IR frameworks (TorchInductor, AI Template, TensorRT, etc.) automate fusion and lowering, often doubling performance via kernel fusion and autotuning. Low-level GPU memory tricks (paged KV caches, async copies, CUDA Graphs, stream overlap) cut idle time and reduce memory overhead, directly improving latency and throughput.

Across all five layers, the highest-leverage wins collapse multiple operations or memory movements into single kernels and keep the GPU fed. Kernel fusion and batch scheduling have immediate large impacts and are now production-ready in many frameworks. Very new research (cluster-level fusion, diffusion-based speculative decoding) shows great promise but remains experimental or niche. Typically, the fastest modern techniques are also the most complex, while simpler approaches (static batching, compiler flags) are easiest to deploy but give smaller gains. By layering these optimizations, current systems have cut inference latencies by multiple factors (up to ~10x in some cases) while preserving exact output quality.

Historical Evolution of Tier-One Inference Optimizations

Speculative Decoding

Speculative Decoding was introduced (Leviathan et al., 2023): a small draft model predicts many tokens in parallel, and the large target model verifies them in bulk. Speculative Sampling and Speculative Prefix Sampling (Chen et al. 2023) studied acceptance rates. In 2023–24, Draft & Verify (Zhang et al., ACL 2024) showed the same LLM can be used as both draft and target by skipping layers (self-speculation). Meta’s LayerSkip (2024) trained LLMs with “early exit” to allow full self-speculation. Other 2024 advances included Lookahead Decoding (Fu et al., 2024), a Jacobi-style predictor achieving ~1.8–4x speedups with multiple GPUs. Diffusion-inspired methods appeared in 2024–25 (DART, SpecDiffusion), where a diffusion model drafts multi-token sequences, achieving up to 7x standard decoding throughput. Tree-based speculative schemes like JetSpec (2026) and Recursive SpecDecoding (2024) explored branching draft generation. The latest (2025–26) focuses on verification algorithms for tree drafts (Traversal Verification).

Attention/Transformer Kernel Optimization

The FlashAttention breakthrough (Dao et al., NeurIPS 2022) fused all attention steps in one kernel, immediately giving ~5–9x speedups on GPUs and linear memory scaling. FlashAttention-2 (ICLR 2024) parallelized along sequence-length; FlashAttention-3 (2024) optimized for Hopper (H100) with warp specialization and asynchronous pipelines. FlashAttention-4 (2026) targeted Blackwell GPUs, yielding up to 1.3x over cuDNN and 2.7x over prior Triton kernels. Sparse Attention research led to kernel-level work: NSA kernels and Flash-Sparse-Attention (ICLR 2026) re-implement sparse patterns, achieving ~1.6x kernel speedup. PagedAttention (Kwon et al., SOSP 2023) applied OS-style paging of KV cache, enabling 2–4x higher throughput in vLLM. Other innovations include block-sparse/local-window kernels, kernel fusion for downstream ops (ClusterFusion fusing QKV+Attn+Projection into one GPU pass), and custom CUDA/Triton kernels.

Runtime Scheduling and Batching

Originally, inference systems did static batching. Dynamic batching (Triton Inference Server, TF Serving) grouped queries arriving close in time. In 2023–24, continuous/dynamic batching took off: vLLM introduced in-flight token batching where new requests join as soon as capacity frees, pumping near-saturation throughput. vLLM’s scheduler also pioneered chunked prefill. FlexLLM (2024) studied token-level scheduling where tokens are scheduled independently across requests. Fairness and priority scheduling (Google’s Duet system) remain mostly custom. Pipeline balancing became crucial: splitting API server from GPU process and pre-computing multiple steps before CPU sync gave 28% throughput gains. CUDA streams and CUDA Graphs cut launch overhead (often automatic in compilers).

Compiler/IR/Graph Optimization

Torch.compile/TorchInductor (PyTorch 2.0) has become a de facto baseline, tracing FX graphs via TorchDynamo, fusing pointwise ops, and lowering to Triton/C++ with autotuning — routinely yielding ~1.8–2x speedups. Meta’s AITemplate (2022) transforms models into template C++/CUDA code, reporting up to 12x speedups vs PyTorch eager on A100. Google’s OpenXLA compiles to optimized kernels, especially for TPU. TensorRT and ONNX Runtime serve as other compiler backends. MLIR-based projects (Torch-MLIR, IREE, TVM) perform graph rewrites, constant folding, dead-code elimination, and layout transformations. Torch.compile is easiest to deploy (one decorator) and robust; TVM/LLVM-based systems offer finer control but require more engineering.

GPU Memory Hierarchy and Execution Optimization

The lowest level of tuning: HBM/DRAM optimization (PagedAttention avoids fragmentation and redundant KV copies); shared memory/registers (custom kernels keep intermediate activations on-chip); memory pooling/allocation (reusing scratch space to reduce cudaMalloc/free calls); pinning and zero-copy for faster CPU→GPU writes; async copies and stream overlap to hide latency; NUMA awareness in multi-GPU servers; L1/L2 caching hints; launch overhead reduction via CUDA Graphs and persistent thread blocks; occupancy tuning; Tensor Core utilization via FP16/FP8/structured sparsity. These memory/workload tricks typically yield tens-of-percent improvement individually, but in aggregate unlock the gains from higher layers.

Complete Taxonomy of Methods

- **Speculative Decoding:** Standard SpecDec (Leviathan 2023), Speculative Sampling (Chen 2023), Draft&Verify (Zhang 2023), LayerSkip (Meta 2024), CLaSp (Chen 2025), SpecPV (Tan 2025), Lookahead (Fu 2024), N-Grammys (Talebi 2024), DART (Liu 2024), SpecDiffusion (Christopher 2025), RSD (Jeon 2024), JetSpec (Hao 2026), Traversal Verify (Lou 2025).
- **Attention/Transformer Kernels:** FlashAttention v1–4 (Dao 2022–26), FlashInfer (Ye 2025), FlashSparse/FSA (Yan 2026), SageAttention (Shen 2024), AITemplate (Meta 2022), Triton kernels, BlockSparse (FlashInfer/PagedAttention), ClusterFusion (Lu 2025), BigBird/Local windows with optimized kernels.
- **Scheduling/Batching:** Static batching, Dynamic batching, Continuous batching (vLLM 2023+), Token-level scheduling (FlexLLM 2024), Priority/fairness schedulers, Chunked prefill, Multi-tenant scheduling, Overlap compute/transfer, CUDA Graph pipelining, Pipeline parallelism.
- **Compiler/Graph:** Torch.compile/Inductor (2023), TorchScript, OpenXLA/XLA, TVM, IREE, Glow, AITemplate, TensorRT, ONNX/ORT, PopTorch/NNEF, MLIR/HLO pipelines.
- **GPU Memory/Execution:** Paged KV Cache (vLLM 2023), CUDA Graphs, pinned buffers, async memcopy, NVLink collective ops, shared-memory KV, memory pooling, persistent kernels, stream concurrency, occupancy tuners, warp specialization, thread block clustering (ClusterFusion 2025).

Detailed Technical Entries

Speculative Decoding Methods

Standard Speculative Decoding (Leviathan et al. 2023): Uses a small, fast draft model to generate a sequence of tokens, then runs the large target model in parallel on that prefix, comparing probabilities to accept or reject each token — guaranteeing the exact same output distribution as standard decoding. Performance: typically ~2–3x throughput, reduces memory traffic by ~30–40% when acceptance is high, no loss of quality. Limitations: gains depend on draft quality and acceptance rate; adds complexity from two-model orchestration. Stability: usable but niche. Obscurity: known to specialists.

Speculative Sampling (Chen et al., 2023): Improves practical acceptance rate via tempered sampling that maximizes acceptance chance, derived from theoretical analysis. More an analysis contribution than standalone method. Stability: research prototype. Obscurity: niche.

Self-Speculative Decoding (Draft & Verify, Zhang et al. 2023): Avoids needing a separate draft model by “skipping” a batch of intermediate transformer layers in the same model to generate draft tokens, then running all layers for verification. Up to ~1.99x speedup on LLaMA-2, no accuracy change. Readiness: production-proven, open-source, simple to integrate. Stability: stable. Obscurity: known to specialists.

LayerSkip (Meta, 2024): Trains the model with random layer-dropout and an “exit loss” during training so a truncated model learns to predict well, enabling stronger self-speculative acceptance at inference. Speedups of 1.34x–2.16x on summarization/coding tasks, near-identical output quality. Requires costly retraining. Stability: usable but niche. Obscurity: niche.

CLaSp (Chen et al., ACL 2025): Dynamic-programming-based adaptive layer-skipping per token, choosing how many layers to skip based on context. 1.3–1.7x speedup on LLaMA-3 variants, <0.5% output change. Stability: experimental prototype. Obscurity: hidden gem.

Speculative Diffusion (SpecDiff, Christopher et al., NAACL 2025): Uses a discrete diffusion model to non-autoregressively sample an entire block of future tokens, verified via standard decoding — a “speculative block” rather than a chain. Up to 7.2x faster than standard decoding, ~1.75x faster than prior spec-decoding methods. Status: research prototype, no widely available code. Obscurity: niche.

DART (Liu et al., ICML 2024): Generates multiple masked tokens in parallel using diffusion-like ideas (Gaussian noise on logits), building a token tree with N-gram constraints. 2.03–3.44x speedup, ~30% better than EAGLE3 baseline. Obscurity: known to specialists.

Lookahead Decoding (Fu et al., 2024): Independently generates candidate tokens for future timesteps and verifies them in parallel using a Jacobi-style update, with no extra training. ~1.8x speedup on conversation tasks, ~4x on codegen with 8 GPUs. Status: research prototype.

JetSpec (Hao et al., 2026): Trains an LLM with a special “tree-causal” attention that generates a full tree of next-token hypotheses in one pass, then verifies via target model. Up to 9.64x on math problems and 4.58x on chat tasks (~1000 tokens/s on a single GPU). Status: experimental/hidden.

Traversal Verification (Lou et al., 2025): In a multi-candidate draft tree, verifies from leaves up (bottom-up) rather than top-down, preserving partial sequences standard verify would drop. 2–6% improvement in acceptance length over conventional verify, exact output distribution guaranteed. Status: research prototype.

Comparisons: Multi-token diffusive methods (SpecDiff, JetSpec) achieve the highest raw speedups but are least mature. Standard spec-decoding is easiest to implement and robust (lossless), gains modest (~2x). Self-speculative (Draft&Verify, LayerSkip) trades minimal retraining for simpler deployment. Lookahead and CLaSp occupy the middle ground. Production systems mostly use standard draft methods since they work on any model without retraining.

Attention / Transformer Kernel Optimization

FlashAttention (Dao et al., NeurIPS 2022): Fuses all attention steps in one kernel with tiled execution, eliminating intermediate writes and making memory usage linear instead of quadratic in sequence length (~10x memory saving at 2K tokens, ~20x at 4K). Up to ~5–9x throughput vs naive matmul-based attention. Stability: production-proven. Obscurity: mainstream.

FlashAttention-2 (Dao, ICLR 2024): Parallelizes over sequence length, allowing multiple thread-blocks to cooperate on one attention call, improving GPU occupancy on large sequences — roughly 2x faster than FlashAttention-1 on certain shapes. Stability: stable. Obscurity: mainstream.

FlashAttention-3 (Shah et al., SYSML 2024): On Hopper GPUs, uses asynchronous (2-CTA) scheduling, specialized warp kernels, and FP8 support — roughly 2x faster than cuBLAS on H100. Stability: stable. Obscurity: mainstream among H100 users.

FlashAttention-4 (Zadouri et al., arXiv 2026): For Blackwell GPUs (B200/GB200), pipelines two MMA per warp, uses fast software math, and larger tile sizes to fully occupy tensor cores. Up to 2.7x faster than Triton and 1.3x over cuDNN 9.13 on BF16, achieving ~71% of peak (1613 TFLOPs on B200). Status: cutting-edge research (2026). Obscurity: hidden gem.

SageAttention Family (Shen et al., 2024-25): Quantized attention kernels (INT8/FP8/INT4/FP4) for consumer GPUs, yielding 1.5–2x speedups by lowering precision. Status: research prototypes, obscurity: deep underground (mostly consumer GPU focus).

FlashInfer (Ye et al., MLSys 2025): Provides highly-optimized kernels plus JIT-generated fusions, block-sparse KV cache format, and dynamic scheduling. 29–69% reduction in inter-token latency vs prior serving compilers; 28–30% better long-context latency; 13–17% faster in parallel-generation. Status: production-ready, integrated into popular inference servers. Stability: stable. Obscurity: known to practitioners.

Paged (Key-Value) Attention (Kwon et al., SOSP 2023): Treats GPU memory like OS “pages,” allocating KV cache in fixed-size blocks that can be shared and swapped across sequences, eliminating fragmentation and duplication. Enables 2–4x throughput improvement over FasterTransformer/Orca at same latency, reduces GPU memory use ~20%. Status: production, core of vLLM engine. Stability: stable. Obscurity: mainstream in inference.

Flash-Sparse / FSA (Yan et al., ICLR 2026): Reorders loops in the kernel to better utilize varying head counts (small GQA groups) on modern GPUs. Up to 3.5x faster kernels than NSA baseline, ~1.25x end-to-end training speed, ~1.36x faster prefill decode. Stability: research prototype. Obscurity: niche.

Flash-Sparse-Attn Library (Shi et al., HKUSTDial): Trainable threshold/gating (drop keys below threshold), local windows, grouped Q attention on top of FlashAttn fusion, handling extremely long contexts. Stability: usable but niche. Obscurity: known to specialists.

ClusterFusion (Luo et al., NeurIPS 2025): Uses NVIDIA Hopper’s cluster shared memory with new primitives (ClusterReduce/Gather) to fuse QKV projection + attention + output-projection in one mega-kernel, eliminating global memory writes between layers. ~1.61x end-to-end latency speedup over best existing frameworks on H100. Stability: experimental. Obscurity: hidden gem.

Comparisons: FlashAttention is easiest to use and usually very fast on NVIDIA GPUs. FlashInfer adds scheduling for an extra 20–50% speedup. AiTemplate is very powerful but requires separate compilation. Sparse kernels help only models explicitly using sparsity. PagedAttention is more about batching/memory. For most LLMs, FlashAttention + memory optimizations gives the best performance today; exotic fusions like ClusterFusion are promising but not mainstream.

Scheduling and Batching Methods

Static Batching: Pre-groups incoming requests by similar input lengths. Simple, stable p50 latency at full utilization; low GPU utilization at low QPS, inflexible to variable arrivals. Stability: stable, production-proven. Obscurity: mainstream.

Dynamic Batching (e.g. Triton): Maintains a queue; forms a batch when enough requests arrive or a timeout hits, padding shorter ones. Higher throughput than static at moderate load; head-of-line blocking can occur. Stability: stable. Obscurity: mainstream.

Continuous/Dynamic Batching (vLLM's approach): Batches continuously — as soon as any slot frees, a new request slots in without resetting the GPU; pre-computes multiple decoding steps in one GPU run (multi-step scheduling) to amortize CPU costs. Achieved ~3.7x higher token throughput than HF's static API in tests, p50 latency 1.5-2x lower at high load. Status: production-proven (vLLM, Orca, MLC). Stability: stable. Obscurity: known to specialists.

Token-Level Scheduling (FlexLLM, Aegaeon): Pools tokens across all requests rather than grouping requests, for very high GPU utilization, but hard to implement (needs "jumping ahead" token support). Status: research prototype. Obscurity: deep underground.

Chunked Prefill (vLLM): Encodes a prompt in chunks so decoding can start earlier and fill the GPU with new tokens, reducing time-to-first-token for long prompts. Status: production. Obscurity: specialist.

Overlapping Compute and Transfer: Uses CUDA streams and pinned memory to copy data concurrently with GPU compute; asynchronous copying keeps the GPU from waiting on host. Can improve throughput by ~5-15% by reducing idle gaps. Stability: production. Obscurity: mainstream concept.

CUDA Graphs and Persistent Kernels: Captures repeated kernel-launch sequences as a CUDA Graph, or uses one persistent kernel that loops to decode many tokens, eliminating launch latency. Reported improvements ~10-20% in end-to-end LLM batching. Status: production (PyTorch 2.x, TensorRT).

Fairness and Priority Scheduling: Assigns weights/priorities to requests, or caps max tokens per request to prevent starvation. Mainly software overhead, reduces tail latency for critical users. Status: implementation-specific.

Comparisons: Continuous batching (vLLM) yields the highest throughput and GPU utilization. Static batching is easiest to deploy but worst on p95 latency. Token-level and overlapping strategies eke out further gains but are complex. Smart chunking and asynchronous graphing are the most overlooked, exploited by few mainstream toolkits by default.

Compiler / IR / Graph Optimization

Torch.compile (TorchInductor): Traces and optimizes graphs via FX, fusing pointwise ops, autotuning block sizes, choosing best GEMM path, and generating Triton/C++ kernels. Reported ~1.8-2x speedups on standard models; user tests show 10-50% per-layer improvement, up to 3.8x for small fused subgraphs. Production readiness: production-proven (PyTorch default in vLLM, Meta's Llama port). Limitations: long warm-up compile time; not all patterns fuse perfectly. Stability: stable. Obscurity: mainstream.

TensorRT (NVIDIA): Imports via ONNX/UFF, optimizes graph (layer merging, kernel selection, precision calibration), runs optimized engines. Often ~2x speedup for supported models (especially INT8). Production: widely used but integration can be heavy (C++ APIs). Stability: stable, production-proven.

OpenXLA / TPU/MLIR (Google): Compiler lowering to HLO/MLIR, originally for TPUs but expanding to CPU/GPU. On GPU, can compete with other frameworks (1-2x). Status: usable on JAX/Flax, emerging for PyTorch.

AI Template (Meta, 2022): Transforms PyTorch models into fused C++ template kernels via vertical/horizontal/memory fusion, relying on FlashAttention for attention. Meta reports up to 12x faster than PyTorch eager on A100 for CNNs and transformers (likely peak-case). Ease: requires exporting model to AIT's format, not drop-in. Stability: stable. Obscurity: known to specialists.

TVM (Apache): Defines compute graph, applies schedule primitives, runs autotuner for best config per hardware; codegen to CUDA/LLVM. Achieves 1.5-2x PyTorch speed with careful tuning. Stability: usable but niche.

IREE (Google): MLIR-based runtime for CPU/GPU, lowering PyTorch/XLA models to MLIR for JIT or AOT compilation. Promising for portable inference, not widely adopted in LLM yet. Stability: experimental.

Generic compiler techniques: Fusion (merging op chains to reduce memory trips), layout transform, autotuning (best GEMM kernel selection), shape specialization (fixed batch/seq sizes), memory planning, dead-code elimination, constant folding.

Comparisons: Torch.compile is easiest (push-button, auto fusion), reliably yielding 1.5-2x. AI Template can yield higher peak (up to 12x in some reports) but needs more engineering. MLIR/TVM/IREE need more setup but allow finer tuning. The most overlooked areas are schedule autotuning and graph partitioning for heterogeneous hardware.

GPU Memory Hierarchy and Execution Tricks

Shared Memory / Register Usage: Custom kernels load Q/K/V fragments into shared memory before compute (FlashAttention-style), keeping intermediate softmax outputs on-chip. Fusing 3 kernels into one changed 5 DRAM round-trips into 2, speeding a Transformer layer by ~3.8x in one teardown. Stability: mainstream practice.

Kernel Launch Reduction (CUDA Graphs / Persistent Kernels): Groups many kernel launches into a graph, or uses a single looping kernel. PyTorch 2.x/TorchInductor auto-applies CUDA Graphs when possible. Multi-step scheduling improved vLLM throughput by 28% on 70B x 4H100 partly via reduced overhead.

CUDA Streams and Concurrency: Uses multiple CUDA streams to overlap memory copies and kernel execution. Reports of continuous overlap improving utilization by 8.7% in one vLLM case.

Occupancy and Warp Specialization: Adjusts thread/block config for high occupancy; warp specialization lets some warps handle shared-memory ops (FlashAttention-3's approach). Can raise occupancy from ~50% to 90% on new architectures.

PCIe/NVLink Optimization: Overlaps GPU-to-GPU communication with computation, essential in multi-GPU tensor-parallel setups; overlapping NCCL with compute can

recover 10–20%.

Memory Pooling / Fragmentation Control: Pre-reserves large GPU buffers for KV cache, shares blocks (vLLM/PagedAttention) to eliminate fragmentation waste, showing ~20% memory saving in the vLLM paper.

Other Low-Level Tricks: Hand-coded PTX/fused ops to reduce register-reuse overhead; casting to FP16/BF16 to use Tensor Cores; pinning threads to CPU sockets near the GPU (NUMA), sometimes worth ~5%.

Comparisons: Shared-memory fusion (FlashAttention-style) gives the single largest wins, multiplying GPU throughput several-fold. Asynchronous execution (streams, graphs) and memory pooling give smaller but vital improvements. SASS/PTX hand-tuning is the least-used, since compilers cover most needs.

Implementation & Hardware Details

Speculative Decoding: Realized by interleaving two forward passes; frameworks like vLLM or FasterTransformer enqueue both models, pipelining to hide the verification pass.

FlashAttention: Triton kernels (Python) or CUDA++ versions; simple drop-in replacement for standard scaled-dot-product-attention calls in PyTorch, or fused XLA ops in JAX/TensorFlow.

FlashInfer: Provides a runtime C++ library and code generation tools, linked for attention/GEMM, scheduling fused operators via JIT.

Torch.compile: Transparent to the user; profiling/caching handles compilation via FX traces, often pre-compiled ahead of serving.

AITemplate: Python front-end API similar to PyTorch, outputs a C++ library; integrates at build time so binaries contain kernels for each model.

GPU tricks: Often implemented in CUDA or Triton; libraries like Megatron-LM implement asynchronous KV-table updates via CUDA streams; C++ APIs use `cudaMemcpyAsync`, `cudaGraphLaunch`, etc.

Production Integrations

- **vLLM:** Uses Torch.compile, FlashAttention, PagedAttention, continuous batching, and CUDA graphs by default — a reference integration of many techniques.
- **NVIDIA Triton / FasterTransformer:** Offers spec-decoding (two-model) and FlashAttention; supports dynamic batching and CUDA graphs.
- **HuggingFace Transformers/TorchScript:** Torch.compile integration; fused kernels for GPU; Triton plugins for attention.
- **SGLang:** Uses FlashInfer, PagedAttention, etc. in its inference library.
- **TensorRT-LLM:** (proprietary) implements many fusions and multi-GPU pipelining.
- **Onnx-MLC/LLaMA.cpp:** Focus on memory (e.g. threaded WASM) more than kernel fusion.

Speculative decoding is just beginning to reach clouds broadly, but FlashAttention and Torch.compile are already default choices in many stacks.

Hidden and Experimental Methods

Less-publicized ideas highlighted: cluster-level primitives (ClusterFusion, exploiting Hopper DSMEM, not in any mainstream library yet); dynamic layer skipping (CLaSp, SPEED, requiring special training logic); speculative tree decodings (Traversals, new algorithms untested at scale); diffusion drafting (SpecDiff, relies on a new model class, hardly used in products); Flash-Sparse and NSA kernels (need sparse-trained models, niche); custom memory allocators (PyTorch caching vs alternatives like jemalloc for pinned memory); shape/time specialization (compiling models per-request shape to eliminate dynamic checks).

Comparative Summary

Area	Method	Speedup	Ease of Deploy	Robustness	Future Potential	Likely Standard
Speculative Decoding	Standard SpecDec	~2-3x	Moderate	High (exact)	Medium	Widely used
Speculative Decoding	Self-Spec (Draft&Verify)	~2x	Easy	High	Medium	Growing
Speculative Decoding	LayerSkip/Early-Exit	~2x	Hard	Medium	Medium	Niche
Speculative Decoding	Lookahead	1.8-4x	Hard	Medium	High	Research
Speculative Decoding	SpecDiffusion	~7x	Hard	Unknown	High	Uncertain
Attention Kernels	FlashAtt v1/v2	5-9x; 10-20x mem	Easy	High	Low (mature)	Standard
Attention Kernels	FlashAtt-3/4	2-3x vs prior	Moderate	High	High	Niche/specialty
Attention Kernels	FlashInfer	13-69% lat reduction	Moderate	High	High	Emerging
Attention Kernels	AITemplate	up to 12x	Hard	High	High	Niche (Meta use)
Attention Kernels	FSA (sparse)	1.6x kernel	Hard	Medium	Medium	Research
Attention Kernels	ClusterFusion	1.61x latency	Very hard	Unproven	High	Unlikely soon
Scheduling	Continuous (vLLM)	~3-4x	Hard	High	High	Likely dominant
Scheduling	Multi-step	+28% on 70B	Moderate	High	Medium	Seen in vLLM
Compiler	Torch.compile	1.8-2x	Very easy	High	Low	Mainstream
Compiler	AITemplate	up to 12x	Hard	High	High	Select use
Memory/Exec	FlashAtt fusion	~3.8x (layer ex.)	Hard	High	Low	Standard
Memory/Exec	Paged KV (vLLM)	2-4x throughput	Hard	High	High	Emerging (vLLM)

Stability and Obscurity Rankings

Speculative Decoding: Standard SpecDec — stable, known to specialists. Self-Spec — stable, known. LayerSkip — experimental, niche. Lookahead — prototype, niche. SpecDiffusion — prototype, deep underground. DART — prototype, niche. JetSpec — prototype, nearly invisible. Traversal Verify — research, known to specialists.

Attention Kernels: FlashAttention 1/2 — production-proven, mainstream. FlashAttention 3/4 — production (v3) / prototype (v4), specialists. FlashInfer — stable,

specialists. AITemplate — stable, specialists. SageAttn — prototype, known to specialists. PagedAttention — stable, known to specialists. FSA/Flash-Sparse — prototype, niche. ClusterFusion — prototype, deep underground.

Scheduling/Batching: Static/Dynamic Batch — stable, mainstream. Continuous (vLLM) — stable, known to specialists. Token-level scheduling — prototype, deep underground. Chunked prefill/multi-step — stable, known to specialists. Fairness/priorities — experimental, niche.

Compiler/Graph: Torch.compile — stable, mainstream. TensorRT — stable, mainstream (NVIDIA stack). AITemplate — stable, specialists. XLA/MLIR/TVM — mature in parts, specialists/non-mainstream. Graph rewriting — stable, mainstream practice.

Memory/Execution: Shared-memory fusion — production, mainstream. Paged KV (vLLM) — stable, known. CUDA Graphs/Streams — production, mainstream. Memory pooling — production, known. PTX/ASM tricks — deep underground.

Research Lineage Maps

Speculative Decoding Map: Traditional decoding → Speculative Sampling (Chen 2023) → Leviathan SpecDec (2022) → Self-Spec (Zhang 2023) → LayerSkip (Meta 2024), CLaSp (2025) → SpecPV (2025) → Lookahead (2024) → Diffusion drafts (DART 2024, SpecDiff 2025) → Cluster/tree decoders (RSD 2024, JetSpec 2026, Traversal 2025). Future likely merges diffusion with tree verification.

Attention Kernel Map: Naive attention → FlashAttention (2022) → FlashA-2,3 (2023–24) → SageAttn quantized (2024–25) → FlashA-4, ClusterFusion (2025–26). Sparse pipelines: BigBird/NSA → Flash-Sparse (2026). AITemplate (2022) fused more ops in parallel; FlashInfer (2025) integrated with scheduling.

Scheduling Map: Simple batching → Triton dynamic batching → Continuous/vLLM batching (2023) → Token-level (FlexLLM 2024). PagedAttention and multi-step scheduling emerged 2023–24. Future: hybrid cloud-edge scheduling and finer LLM “kernel” scheduling.

Compiler Map: Static graph (TorchScript) → Torch.compile (Inductor) (2023) → Fusion frameworks (AITemplate 2022, TensorRT) → MLIR-based (OpenXLA, IREE). PyTorch 2.x consolidated many features; future likely brings more dynamic graph compilers and domain-specific IRs.

Memory/Exec Map: GPU mem pooling → Paged KV (2023) → Unified KV sharing (2024). Overlap (streams) is old; CUDA Graphs for LLM specifically (2023+). ClusterFusion (2025) uses new GPU features; upcoming GPU features expected to open more tricks.

Future Directions

- **Unified Execution Engine:** Combining many techniques — an inference engine that automatically applies Torch.compile and speculative decoding with optimal draft selection per query.
- **Adaptive Hybrid Strategies:** Dynamically switching between methods per workload (e.g. spec-decoding only if acceptance likely).
- **LLM-Designed Hardware Blocks:** Hardware primitives for speculative parallelism (future GPUs might expose an “LLM shader”).

- **Memory-Compute Co-design:** Closer integration of cluster-level comms (like ClusterFusion) into CUDA or Transformer libraries.
- **Differentiable Compilers:** Auto-tuning by RL or genetic algorithms to find best optimizations for a given model.
- **Streaming / Edge Inference:** Applying these techniques at small batch sizes or on smaller devices — hybrid CPU/GPU schedules or even FPGAs with custom streams.

Comprehensive Bibliography

Selected key sources referenced throughout this report include: Leviathan et al., “Fast Inference from Transformers via Speculative Decoding,” ICLR 2023; Chen, J.X., et al., “Speculative Sampling,” NAACL 2023; Zhang, J., et al., “Draft & Verify,” ACL 2024; Tan, W., et al., “SpecPV,” ICML 2025; Fu, Z., et al., “Lookahead Decoding,” arXiv 2024; Christopher, C., et al., “Speculative Diffusion Decoding,” NAACL 2025; Liu, Z., et al., “DART,” ICML 2024; Luo, X., et al., “ClusterFusion,” NeurIPS 2025; Dao, T., et al., “FlashAttention” series, NeurIPS 2022 / ICLR 2024 / arXiv 2026; Ye, Z., et al., “FlashInfer,” MLSys 2025; Kwon, W., et al., “Efficient Memory Management for LLM Serving with PagedAttention,” SOSP 2023; Yan, R., et al., “FSA,” ICLR 2026; Meta AI, “AITemplate,” blog 2022; HuggingFace/vLLM engineering blogs on Torch.compile and performance updates.

LocalHouseLLM Research — Internal Technical Compendium

Document prepared by LocalHouseLLM Research. All findings, ratings, and citations herein reflect internal review of primary and secondary sources current as of the report date. For internal / partner distribution.